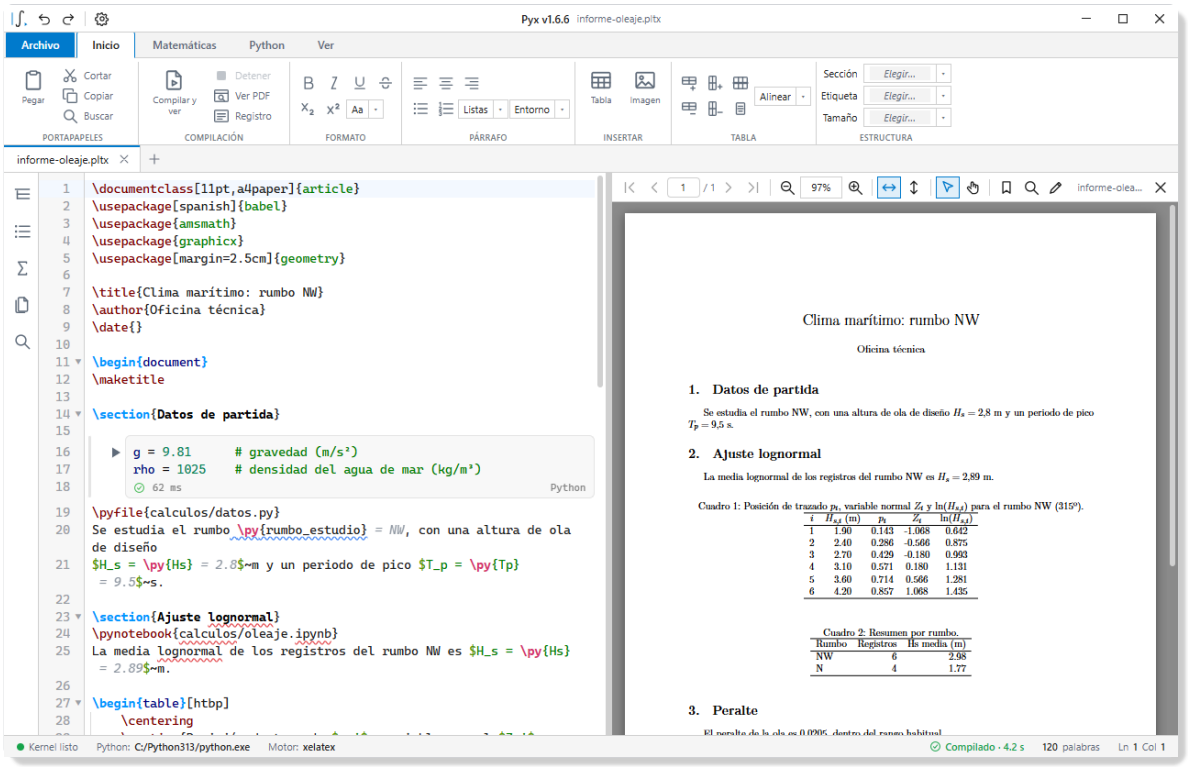
Pyx

Manual de usuario

LaTeX y Python en un mismo documento:
escribe el informe, calcula en él y obtén el PDF.



Versión 2.0.0
noviembre de 2026

Windows · macOS · Linux
Escrito y compilado con Pyx

Pyx — Manual de usuario

Versión 2.0.0 · 14 de noviembre de 2026

Este manual se ha escrito con Pyx y se ha compilado con Pyx: es un documento `.pltx` con celdas de Python que se ejecutaron al compilarlo, con Python 3.13.7 y NumPy 2.3.4. La figura 7.1 y los valores numéricos del capítulo 7 los calculó una de esas celdas; ninguno se escribió a mano.

Pyx es software libre, publicado con la licencia MIT. Código fuente, instaladores y registro de cambios: `https://github.com/LorGIOO/Pyx`.

Las capturas de pantalla se tomaron en Windows; en macOS y Linux la aplicación es idéntica, salvo los botones de la ventana y las rutas de los archivos.

Índice general

1 Bienvenida **5**

1.1 Qué hace Pyx 5

1.2 Novedades de la versión 2.0 6

1.3 Cómo leer este manual 6

2 Instalación y primer arranque **7**

2.1 Requisitos 7

2.2 Instalar Pyx 7

2.3 Actualizar Pyx 7

2.4 Preparar Python y LaTeX con un clic 8

2.5 Elegir el intérprete de Python 9

2.6 La pantalla de inicio 9

3 Recorrido por la interfaz **10**

3.1 La cinta 11

3.2 El panel lateral 11

3.3 Temas 12

4 Documentos y proyectos **13**

4.1 Crear un documento 13

4.2 El formato `.pltx` 13

4.3 Abrir, guardar y cerrar 14

4.4 Proyectos de varios archivos 14

4.5 Dónde guarda Pyx lo que no es tuyo 14

5 Escribir LaTeX **15**

5.1 Autocompletado 15

5.2 La cinta de formato 16

5.3 Tablas 16

5.4 Imágenes y figuras 18

5.5 Fórmulas: vista previa al pasar el ratón 19

5.6 Navegar por el documento 19

5.7 Edición inteligente 20

5.8 Comprobaciones mientras escribes . . . 20
5.9 Ortografía, puntuación y redacción . . . 20
5.10 Buscar y reemplazar . . . 21

6 Celdas de Python 23
6.1 Crear y ejecutar celdas . . . 23
6.2 Las salidas . . . 23
6.3 Errores . . . 24
6.4 Autocompletado y ayuda de Python . . . 25
6.5 El kernel . . . 26

7 Del cálculo al documento 27
7.1 Un valor en el texto: `\py{}` . . . 27
7.2 Un texto u otro: `\pyif` . . . 28
7.3 Tablas y figuras completas: el módulo `pyx` . . . 28
7.4 Un ejemplo completo . . . 28
7.5 El desarrollo del cálculo: handcalcs . . . 29

8 Cuadernos de Jupyter y archivos de Python 30
8.1 Qué se puede usar . . . 30
8.2 Sin volver a ejecutarlos . . . 30
8.3 Se vigilan mientras trabajas . . . 31
8.4 Archivos `.py` y sus celdas . . . 31
8.5 Varios archivos y variables con el mismo nombre . . . 31

9 Ejecución reactiva 32
9.1 Qué se ejecuta . . . 32
9.2 Celdas desactualizadas . . . 32
9.3 Mayúsculas+Intro, reactivo . . . 33
9.4 Garantías . . . 33

10 Compilar 35
10.1 Las formas de compilar . . . 35
10.2 Rápido: el motor precalentado . . . 35
10.3 Figuras y tablas en su sitio . . . 36
10.4 Motores . . . 36
10.5 Bibliografía, índices y glosarios . . . 36
10.6 El registro y los problemas . . . 37

11 El visor de PDF **38**

11.1 La barra del visor 38

11.2 Rápido con documentos enormes 38

11.3 Revisar y corregir el PDF 39

11.4 Del código al PDF y del PDF al código (SyncTeX) 39

11.5 Marcadores 40

12 Macros en Python **41**

13 Configuración **43**

14 Atajos de teclado **45**

15 Solución de problemas **47**

16 Glosario **50**

Créditos **51**

1

Bienvenida

Pyx es un editor de documentos LaTeX con celdas de Python dentro. En el mismo archivo escribes el texto del informe, haces los cálculos y colocas sus resultados en el texto; al compilar, Pyx ejecuta las celdas, sustituye cada valor en su sitio y genera el PDF. Si cambias un dato, todo lo que depende de él se actualiza solo: el número del párrafo, la tabla, la gráfica y la conclusión.

Está pensado para quien escribe documentos técnicos con números dentro (memorias de cálculo, informes de laboratorio, trabajos académicos, anejos de proyecto) y está cansado de copiar resultados de una hoja de cálculo o de Jupyter al documento, y de descubrir después que el PDF y los cálculos ya no coinciden.

1.1 Qué hace Pyx

Un editor de LaTeX completo. Resaltado, autocompletado de más de 300 comandos y entornos, comprobación de errores mientras escribes, ortografía y gramática, plegado, marcadores y búsqueda en todo el proyecto: lo que esperas de TeXstudio.

Celdas de Python, como en Jupyter. Bloques de código dentro del documento, con su salida debajo: texto, tablas, gráficas, fórmulas. Autocompletado real con jedi y ayuda de cada función.

Tus cuadernos y scripts, sin repetir el cálculo. Un cuaderno de Jupyter (`.ipynb`) o un archivo `.py` se vinculan al documento y sus variables se usan igual que las de una celda. Si el cuaderno está abierto en VS Code o Jupyter, Pyx toma sus valores de allí al momento, sin ejecutar nada.

Un puente del cálculo al texto. `\py{expresión}` escribe en el PDF el valor de una expresión de Python, y `\pyif` elige un texto u otro según el resultado.

Un visor de PDF integrado. El PDF se ve al lado del código, con salto en los dos sentidos (Ctrl+clic), búsqueda, marcadores y anotaciones.

Rapidez. Pyx recalcula solo las celdas afectadas por un cambio y mantiene LaTeX con el preámbulo ya cargado, así que el PDF se actualiza en décimas de segundo mientras escribes.

1.2 Novedades de la versión 2.0

- **Compilar es más rápido**: Compilar y ver aprovecha el motor de LaTeX con el preámbulo ya cargado y el PDF se escribe con una compresión ligera. En un informe real de 18 páginas, cada pasada baja de 17 s a 13 s (capítulo 10).
- **El visor tiene las páginas listas antes de llegar a ellas**: mientras la vista está quieta dibuja el resto del documento, y desplazarse es instantáneo. La página se mueve arrastrando con el **botón derecho** (capítulo 11).
- **Anotaciones de verdad**: marcas que se editan con tiradores, flechas rectas, curvas o en codo, grosor a medida, puntero circular del tamaño de la punta, borrador por partes, cuadros de texto, láser, comentarios plegables y exportar el PDF con las marcas (sección 11.3).
- **Tus carpetas, limpias**: un `.pltx` guarda solo tu documento y sus resultados, y la carpeta de trabajo de Pyx no acumula copias (sección 4.2).
- **Tablas e imágenes**: asistentes nuevos con vista previa del código, pegar desde Excel en una rejilla y un **estilo propio** que se guarda (secciones 5.3 y 5.4).
- **Ctrl+clic exacto** entre el código y el PDF, también debajo de tablas y figuras generadas con Python.
- **Corrector más fino**: rojo para la ortografía, azul para la puntuación y los espacios, verde para la redacción (sección 5.9).
- **Las figuras no se adelantan a su apartado** (sección 10.3).
- **Actualizar con un botón**: **Configuración** → **General** → **Buscar actualizaciones** descarga, instala y reabre Pyx solo (sección 2.3).
- Ctrl + R reemplaza la palabra seleccionada en todo el documento.

1.3 Cómo leer este manual

Los capítulos 2 a 4 explican cómo instalar Pyx, cómo es la ventana y cómo se trabaja con documentos. Los capítulos 5 a 9 recorren lo que haces a diario: escribir LaTeX, programar en celdas, usar tus cuadernos de Jupyter y archivos de Python, y llevar los resultados al texto. A partir del capítulo 10 se explican la compilación, el visor de PDF y las herramientas de apoyo. Al final hay una tabla con todos los atajos de teclado (capítulo 14) y un capítulo de solución de problemas.

A lo largo del texto encontrarás tres tipos de recuadro:

Consejo Una forma más rápida o más cómoda de hacer algo.

Nota Un detalle que conviene conocer para entender lo que hace Pyx.

Atención Algo que puede dar un resultado inesperado si no se tiene en cuenta.

Las teclas aparecen así: Ctrl + S. En macOS, Ctrl es Cmd. Los botones de la aplicación aparecen así: Compilar y ver.

2

Instalación y primer arranque

2.1 Requisitos

- **Windows** 10 u 11 (64 bits), **macOS** 11 o posterior (Intel o Apple Silicon) o **Linux** de 64 bits (Ubuntu 22.04 o equivalente).
- 4 GB de memoria y unos 600 MB de disco si Pyx prepara su propio Python, o 200 MB si usa uno que ya tengas.
- Conexión a Internet solo la primera vez, si Pyx tiene que preparar Python o LaTeX, y para buscar actualizaciones.

No necesitas instalar nada más. Si ya tienes Python (Anaconda, python.org, Microsoft Store…) o una distribución de LaTeX (MiKTeX, TeX Live, MacTeX), Pyx los encuentra y los usa. Si no, te ofrece prepararlos.

2.2 Instalar Pyx

Descarga el instalador de tu sistema desde la página de versiones (`https://github.com/LorGIOO/Pyx/releases`). Junto a los instaladores de cada versión está también este manual, en PDF, de esa misma versión.

Windows: `Pyx_2.0.0_x64-setup.exe`. Ábrelo y sigue el asistente; no pide permisos de administrador. Al terminar, los archivos `.pltx` (y, si quieres, los `.tex`) se abren con Pyx al hacer doble clic.

macOS: `Pyx_2.0.0_universal.dmg`, válido para Mac con Intel y con Apple Silicon. Arrastra Pyx a Aplicaciones. La primera vez, abre la aplicación con clic derecho → **Abrir**.

Linux: `.deb` (Ubuntu, Debian), `.rpm` (Fedora) o `.AppImage` (cualquier distribución: dale permiso de ejecución y ábrelo).

2.3 Actualizar Pyx

Pyx se actualiza solo, como VS Code. En **Configuración** → **General** → **Actualizaciones** está la versión instalada y el botón `Buscar actualizaciones`. Al pulsarlo, si hay una versión nueva, Pyx la descarga, comprueba que es idéntica a la publicada (su huella SHA-256), la instala y se vuelve a abrir. Solo te pregunta si hay documentos sin guardar, porque la instalación cierra la aplicación un momento. Además, unos segundos después de arrancar Pyx lo comprueba por su cuenta y, si hay novedades, la barra de estado ofrece `Actualizar a…`.

La versión que tienes se ve siempre en la barra de título y en el nombre de la ventana (en la barra de tareas y en Alt + Tab). En macOS y Linux el botón abre la página de la versión para descargarla.

Si el antivirus bloquea el instalador Los instaladores de Pyx no llevan firma digital de pago, y algún antivirus puede desconfiar de una versión recién publicada. Antes de publicar cada versión se comprueban con Kaspersky. Si aun así tu antivirus lo bloquea, permite el archivo en su lista de excepciones o vuelve a intentarlo pasados unos minutos.

2.4 Preparar Python y LaTeX con un clic

Al arrancar, Pyx comprueba si hay un intérprete de Python y un motor de LaTeX. Si falta alguno, aparece el diálogo **Preparar Pyx** (figura 2.1).

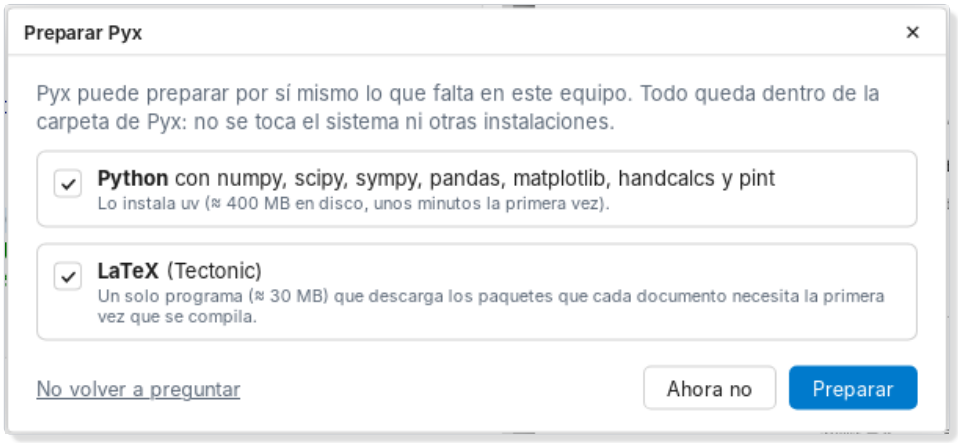


Figura 2.1: El diálogo «Preparar Pyx» aparece cuando falta Python o LaTeX.

Python. Pyx descarga *uv*, un instalador de Python muy rápido, y con él un Python 3.12 independiente con las librerías que usa un informe de cálculo: NumPy, SciPy, SymPy, pandas, Matplotlib, handcalcs, Pint, openpyxl, IPython y jedi. Tarda entre uno y cinco minutos según la conexión, y ocupa unos 400 MB.

LaTeX (Tectonic). Un único programa de unos 30 MB que, la primera vez que compila un documento, descarga solo los paquetes de LaTeX que ese documento necesita.

Pulsa Preparar. Verás en directo lo que se descarga e instala. Al terminar, Pyx pasa a usar ese Python y ese LaTeX.

Dónde se instala Todo queda dentro de la carpeta de datos de Pyx: %LOCALAPPDATA%\com.pyx.studio en Windows, ~/Library/Application Support/Pyx en macOS y ~/.local/share/pyx en Linux. No se modifica el PATH ni se toca ninguna otra instalación. La carpeta del programa (%LOCALAPPDATA%\Pyx) contiene solo el programa: las compilaciones, los registros y los entornos de Python van siempre a la carpeta de datos. Al desinstalar Pyx, el desinstalador ofrece borrarla también.

Si ya tienes Python y LaTeX, el diálogo no aparece. Si lo cierras con Ahora no, puedes abrirlo más tarde desde **Configuración** → **Celdas Python** → **Python de Pyx** o **Configuración** → **Compilación** → **Descargar Tectonic**. No volver a preguntar evita que aparezca al arrancar.

2.5 Elegir el intérprete de Python

Pyx usa, por este orden:

1. el intérprete que hayas elegido tú (**Python** → **Intérprete**, o **Configuración** → **Celdas Python** → **Ruta de Python**);
2. el Python que haya preparado el propio Pyx;
3. un Python del sistema que tenga NumPy;
4. cualquier otro Python que funcione.

La barra de estado indica siempre el intérprete en uso. El desplegable Intérprete de la pestaña **Python** de la cinta lista todos los que Pyx encuentra en el equipo (junto a cada uno se indica si tiene NumPy) y permite elegir un `.exe` concreto, por ejemplo el de un entorno de Anaconda.

2.6 La pantalla de inicio

Sin ningún documento abierto, Pyx muestra la pantalla de bienvenida (figura 2.2). Desde ella creas un documento nuevo (Ctrl + N) o abres uno existente (Ctrl + O). Un archivo `.pltx` o `.tex` también se abre haciendo doble clic sobre él en el explorador de archivos.

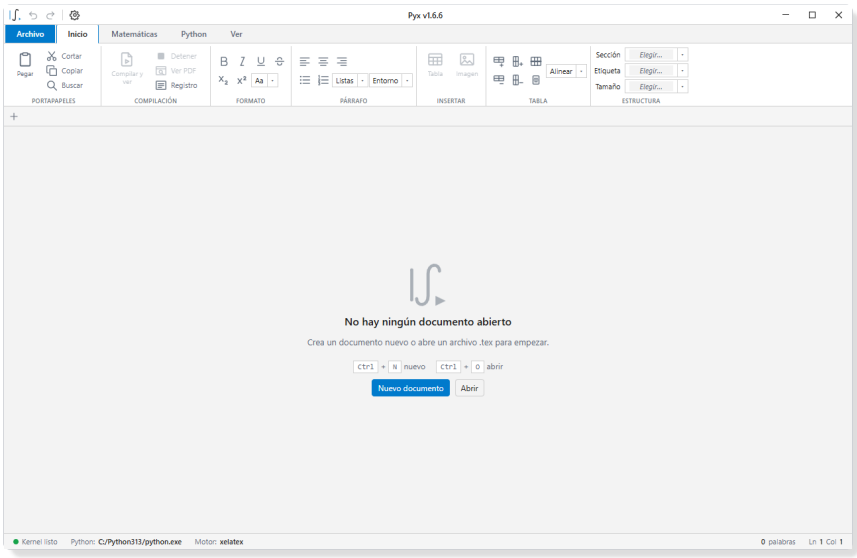


Figura 2.2: La pantalla de inicio, sin documentos abiertos.

3

Recorrido por la interfaz

La figura 3.1 muestra la ventana de Pyx con un informe abierto. Cada número corresponde a una zona que se explica a continuación.

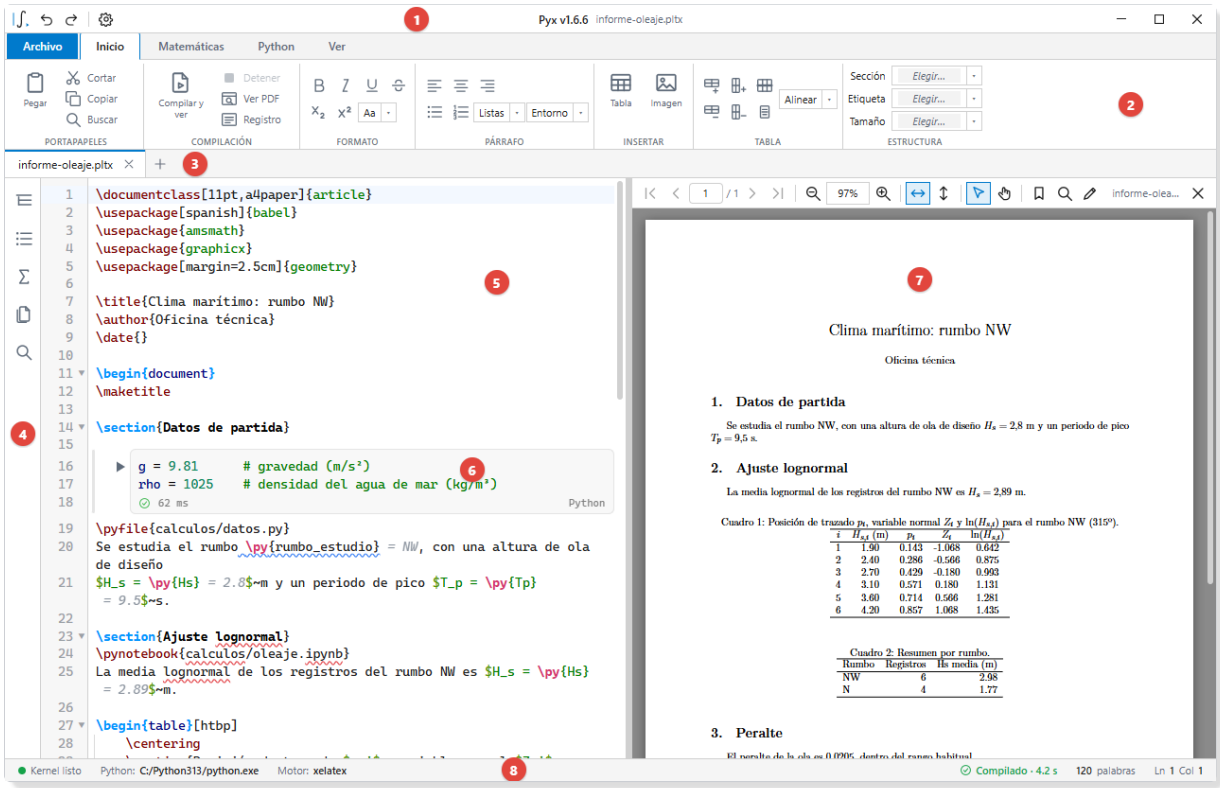


Figura 3.1: La ventana de Pyx: las ocho zonas principales.

1. **Barra de título.** La versión de Pyx, el nombre del documento activo (un punto delante indica cambios sin guardar), los botones de deshacer y rehacer y el engranaje de **Configuración**.
2. **Cinta.** Las herramientas, agrupadas en pestañas: **Archivo**, **Inicio**, **Matemáticas**, **Python** y **Ver**. Se describen en la sección 3.1. Como en Word, se adapta al ancho de la ventana: con poco sitio los grupos se compactan y, al final, se recogen en un botón que los despliega.
3. **Pestañas de documentos.** Un documento por pestaña; `+` crea uno nuevo. Las imágenes y páginas web que generan las celdas también se pueden abrir como pestañas.
4. **Panel lateral.** Una tira de iconos que abre los paneles de **Estructura**, índice (**TOC**), **Símbolos**, **Archivos** y **Buscar en el proyecto**.
5. **Editor.** El código LaTeX del documento, con números de línea, plegado y resaltado. Junto a cada `\py{}` se ve en gris su valor actual.
6. **Celda de Python.** Un bloque de código con su barra de herramientas y su salida debajo.

7. **Visor de PDF.** El documento compilado, al lado del código. Su barra de herramientas está en la sección 11.

8. **Barra de estado.** El estado del kernel de Python (verde: listo; ámbar: ejecutando; rojo: error), el intérprete y el motor de LaTeX en uso, el resultado de la última compilación, el número de palabras y la posición del cursor. Si el documento importa librerías que el Python en uso no tiene, aparece aquí un aviso **!** Falta: … que permite instalarlas con un clic.

3.1 La cinta

Archivo Nuevo, Abrir, Guardar, Guardar como y Cerrar.

Inicio *Portapapeles* (pegar, cortar, copiar, buscar); *Compilación* (Compilar y ver, Detener, Ver PDF y Registro, como en TeXstudio); *Formato* (negrita, cursiva, subrayado, tachado, subíndice, superíndice, mayúsculas y minúsculas); *Párrafo* (alineación, listas, entornos); *Insertar* (asistentes de Tabla e Imagen); *Tabla* (añadir y quitar filas y columnas, alinear); *Estructura* (secciones, etiquetas y tamaños de letra).

Matemáticas Modo en línea o independiente, subíndices, superíndices, raíces, fracciones, delimitadores que se ajustan, entornos de ecuaciones, funciones, estilos, operadores, acentos y espacios.

Python Nueva celda; Celda actual y Todas las celdas (que pasa a Interrumpir mientras algo se ejecuta); minimizar o expandir todas las celdas; reiniciar el kernel, limpiar y ocultar salidas; las Macros; el Intérprete y el estado del kernel.

Ver Dividir el editor, cerrar paneles, modo zen, vista previa, panel lateral, registro de compilación y terminal.

3.2 El panel lateral

Estructura El archivo actual y los que incluye con `\input`, `\include` o `\subfile`; un clic los abre.

TOC El índice del documento (partes, capítulos, secciones…), con la sección del cursor resaltada, y la lista de tareas pendientes (`% TODO`, `\todo`). Un clic lleva a la línea.

Símbolos Letras griegas, operadores, relaciones, flechas y demás símbolos matemáticos. Un clic los inserta.

Archivos Un explorador de la carpeta del documento: abrir, crear, renombrar y borrar archivos y carpetas.

Buscar en el proyecto Búsqueda y reemplazo en todos los archivos (sección 5.10.1).

Ctrl + Alt + Z activa el **modo zen**: desaparecen la cinta, el panel lateral y la barra de estado, y quedan solo el editor y el PDF. Un punto de color junto a las pestañas indica la compilación (gris: sin compilar; azul: LaTeX; amarillo: Python; verde: correcto; ámbar: con avisos; rojo: error); un clic lleva al primer error y Ctrl + Alt + L abre el registro. Esc vuelve a la vista normal.

3.3 Temas

Pyx tiene tres temas: **Claro**, **Oscuro** y **Azul**. Se eligen en **Configuración** → **General** → **Tema** y se aplican al momento, también a la ventana auxiliar del visor. El botón de contraste del visor oscurece además el fondo de las páginas del PDF para trabajar de noche.

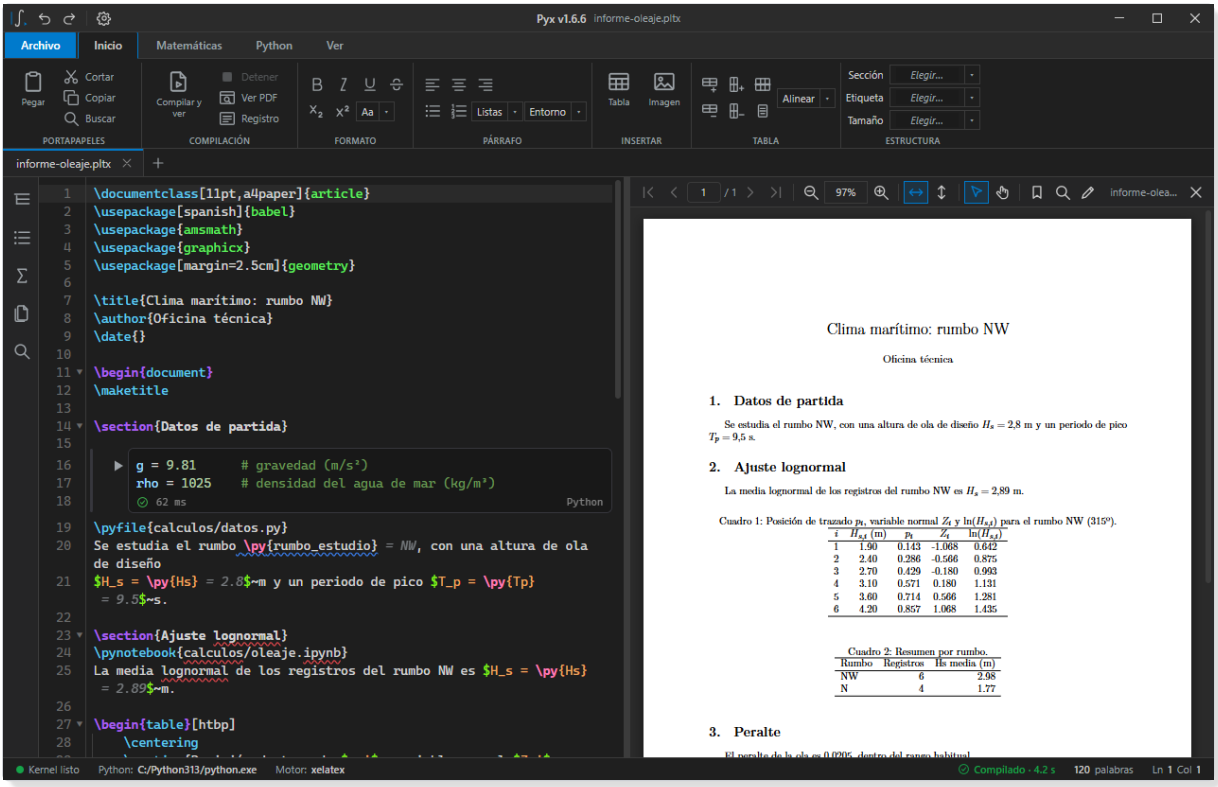


Figura 3.2: El tema oscuro.

4

Documentos y proyectos

4.1 Crear un documento

Nuevo (Ctrl+N) pregunta qué tipo de documento quieres (figura 4.1):

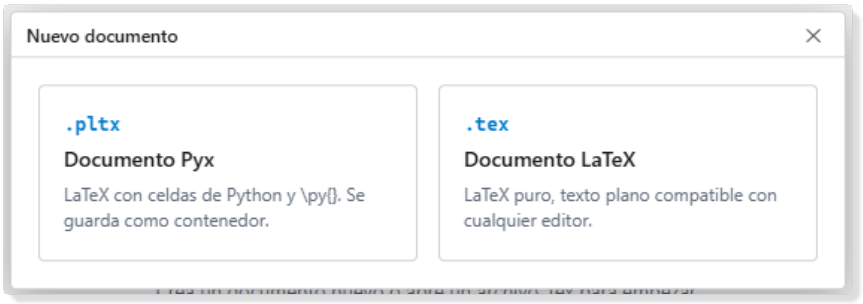


Figura 4.1: Los dos tipos de documento.

Documento Pyx (`.pltx`). LaTeX con celdas de Python y `\py{}`. Es el formato propio de Pyx.

Documento LaTeX (`.tex`). LaTeX puro, en texto plano, compatible con cualquier editor.

4.2 El formato `.pltx`

Un `.pltx` es un contenedor (un archivo ZIP, como los `.docx` de Word) que guarda solo dos cosas:

- el código del documento, LaTeX con sus celdas;
- los **resultados de las celdas**: al reabrir el documento se ven los números, las tablas y las figuras sin volver a ejecutar nada, incluso en un equipo sin Python.

Por eso un `.pltx` ocupa lo que tu texto: unos pocos kilobytes. El PDF y los archivos auxiliares de LaTeX no se guardan dentro ni junto al documento: viven en la carpeta de trabajo de Pyx (sección 4.5). Para sacar una copia del PDF (para enviarla o imprimirla) usa el botón Guardar una copia del PDF del visor.

> **Nota** Dentro de un `.pltx` las celdas son comentarios de LaTeX (`%#python` y `%#end`), así que el código sigue siendo LaTeX válido. Si añades celdas a un `.tex`, al guardar Pyx te propone guardarlo como `.pltx`.

4.3 Abrir, guardar y cerrar

- Abrir (Ctrl + O) abre `.pltx`, `.tex`, `.sty`, `.cls`, `.bib` y `.txt`. Los archivos `.py`, las imágenes y los PDF se abren desde el panel **Archivos**: el código en el editor, las imágenes en una pestaña y los PDF en el visor.
- Guardar (Ctrl + S) guarda el documento activo, lo compila y muestra el PDF.
- Guardar como cambia el nombre o la carpeta. Pyx avisa si el documento tiene cambios sin guardar al cerrar su pestaña (Ctrl + W) o la aplicación.
- Los archivos incluidos que no tienes abiertos se leen del disco en cada compilación: lo que cambies en ellos con otro programa entra en el siguiente PDF.

4.4 Proyectos de varios archivos

Un informe largo se suele dividir en un documento principal y varios capítulos que este incluye con `\input{capitulo1}` o `\include{capitulo1}`. Pyx entiende esa estructura:

- al compilar un capítulo, compila el documento principal que lo incluye;
- las celdas de todos los archivos se ejecutan en el orden en que aparecen en el documento completo;
- las referencias, citas y etiquetas de cualquier archivo abierto se ofrecen al autocompletar;
- el panel **Estructura** muestra los archivos incluidos, y **Buscar en el proyecto** recorre todos.

4.5 Dónde guarda Pyx lo que no es tuyo

Para compilar, LaTeX necesita una copia del documento con los valores de Python ya puestos y genera archivos auxiliares (`.aux`, `.log`, el índice de SyncTeX, el PDF). Pyx los guarda en su **carpeta de trabajo**, dentro de su carpeta de datos (%LOCALAPPDATA%\com.pyx.studio), nunca en la carpeta de tu proyecto ni en la del programa. Tu carpeta contiene solo lo que tú pones en ella.

Esa carpeta de trabajo se mantiene sola: el visor no duplica el PDF, las carpetas de los proyectos que no compilas desde hace un mes se borran al arrancar Pyx (la próxima compilación las vuelve a crear) y los archivos intermedios que no se reutilizan se eliminan.

5

Escribir LaTeX

5.1 Autocompletado

Al escribir una barra invertida, Pyx propone los comandos que empiezan por lo que llevas escrito, con una breve descripción de cada uno (figura 5.1). Con ↑ ↓ eliges y con `Intro` o `Tab` aceptas. Los comandos con argumentos se insertan con los huecos ya puestos: `\frac{num}{den}` deja seleccionado *num*; `Tab` salta a *den*.

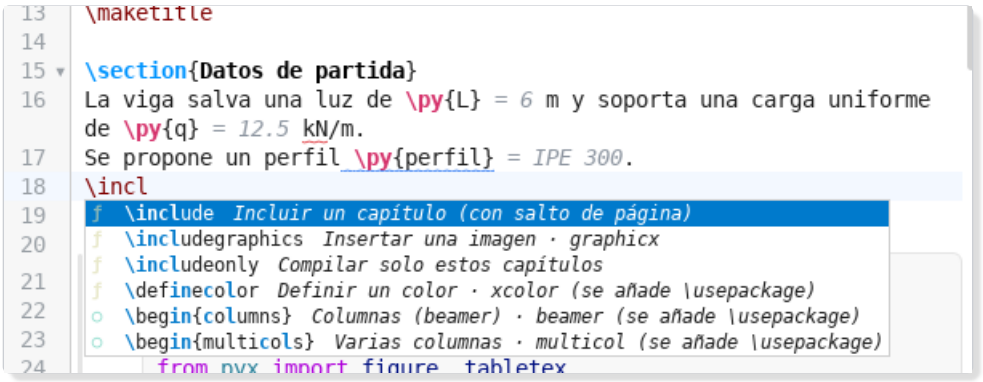


Figura 5.1: Autocompletado de comandos: descripción y paquete de cada uno.

Paquetes automáticos. Si aceptas un comando de un paquete que el documento no carga (por ejemplo `\qty`, de `siunitx`), aparece «(se añade `\usepackage`)» y, al aceptarlo, Pyx añade `\usepackage{siunitx}` al preámbulo.

Tus comandos. Los comandos que define el propio documento con `\newcommand`, `\DeclareMathOperator` o `\def` se ofrecen también, con su número de argumentos.

Entornos. Tras `\begin{` se listan los entornos (figura 5.3). Al elegir uno, Pyx escribe también el `\end` y el contenido habitual: una tabla trae su especificación de columnas; una figura, su `\includegraphics`, su `\caption` y su `\label`. La primera opción, «{ … }», sirve para escribir el nombre de un entorno propio.

Referencias, citas y glosario. Dentro de `\ref{`, `\eqref{`, `\cref{`… se ofrecen las etiquetas de todos los documentos abiertos; dentro de `\cite{`, las claves de los `.bib` que cargas; dentro de `\gls{`, las entradas del glosario y las siglas.

Valores de Python. Al escribir `\py{` se abre la lista de las variables que hay (figura 5.2): las que definen las celdas del documento y las de los cuadernos y archivos `.py` que vincula, con el archivo del que viene cada una. Funciona en cualquier punto de la expresión (`\py{filas(rumbo_` también completa) y, tras un punto, ofrece los atributos del valor.

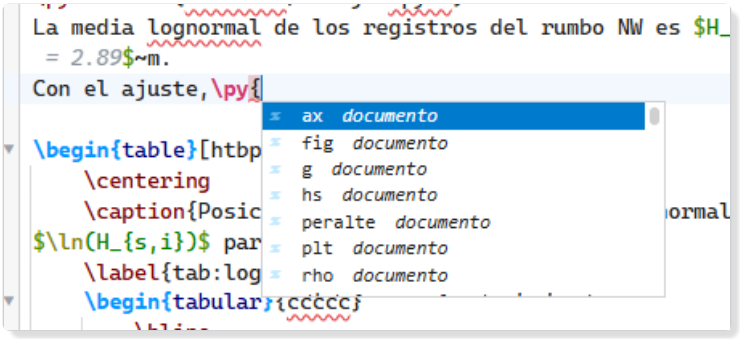


Figura 5.2: Dentro de `\py{`: las variables del documento, del cuaderno y del archivo `.py`.

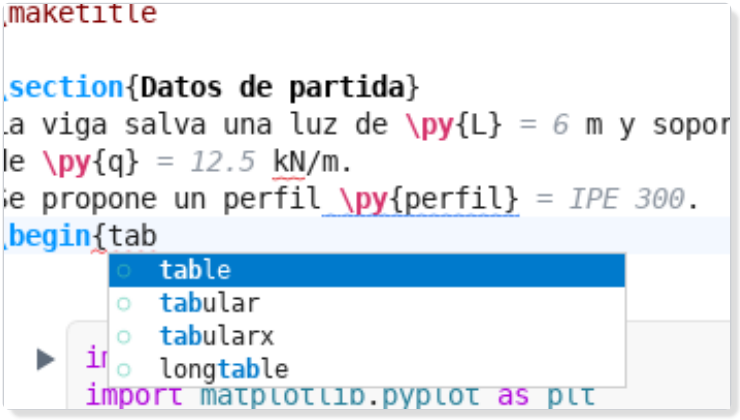


Figura 5.3: Al escribir `\begin{tab` se listan los entornos de tabla.

Consejo El autocompletado se ajusta en **Configuración** → **Autocompletado**: si aparece solo al escribir o únicamente con `Ctrl`+`Espacio`, si distingue mayúsculas y si muestra iconos.

5.2 La cinta de formato

La pestaña **Inicio** aplica formato a la selección (o deja el cursor dentro, si no hay selección): **negrita** (`Ctrl`+`B`), *cursiva* (`Ctrl`+`I`), subrayado, tachado, subíndice, superíndice y cambio de mayúsculas. El grupo *Estructura* inserta secciones (con su nivel indicado: Pt, Cap, S1, S2…), etiquetas, referencias y tamaños de letra. La pestaña **Matemáticas** hace lo mismo con las fórmulas.

5.3 Tablas

5.3.1 El asistente de tabla

Tabla (pestaña **Inicio**) abre el asistente (figura 5.4). La tabla se rellena en una rejilla, como una hoja de cálculo: escribe en las celdas, o copia un bloque de Excel y pégalo en cualquier celda con `Ctrl`+`V`, y la rejilla crece sola para que quepa. `Intro` baja a la fila siguiente (y la crea si hace falta).

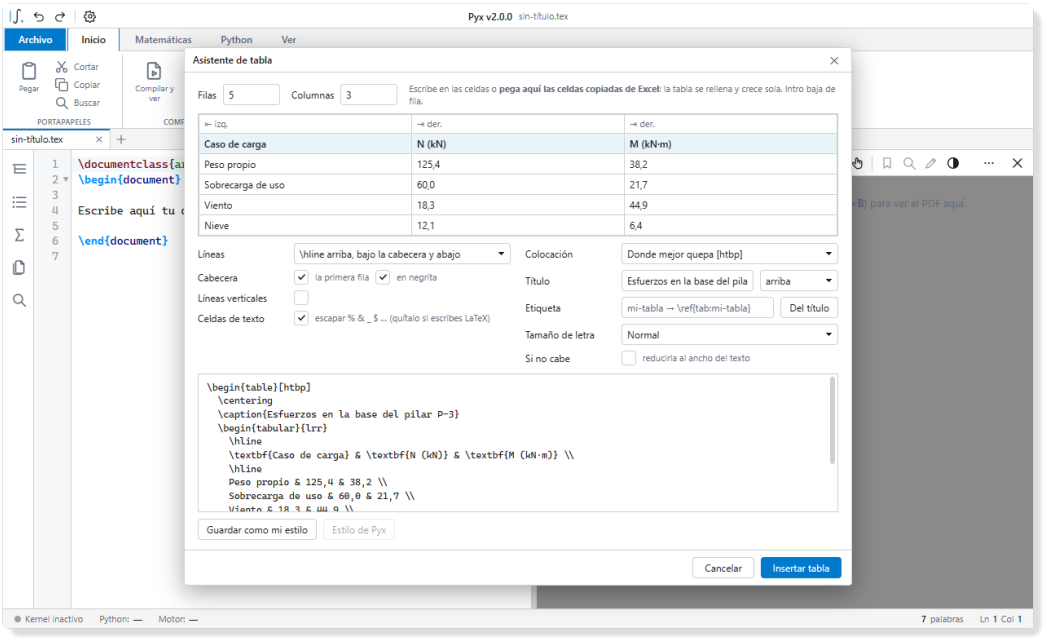


Figura 5.4: El asistente de tabla con cuatro filas pegadas desde Excel y el código que va a insertar.

Columnas. Cada una con su alineación: izquierda, centro, derecha o párrafo (`p{ancho}`, para texto que se parte en varias líneas). Al pegar, las columnas de números se alinean a la derecha solas.

Líneas. Cuatro estilos: `booktabs` (el profesional, sin líneas verticales), `\hline` arriba, bajo la cabecera y abajo, cuadrícula completa o sin líneas.

Cabecera. La primera fila, con una línea debajo y, si quieres, en negrita.

Celdas de texto. Escapa %, &, _, $… Desactívalo si escribes LaTeX en las celdas (`$x^2$`).

Colocación, título y etiqueta. El entorno `table` con su `\caption` arriba o abajo y su `\label`; Del título propone la etiqueta a partir del título. «Solo el tabular» inserta la tabla sin entorno flotante.

Tamaño de letra y «si no cabe». Letra más pequeña para tablas anchas, o reducir la tabla al ancho del texto solo cuando no cabe.

Debajo se ve exactamente el código que se insertará y, si hace falta, el paquete que debe cargar el preámbulo.

5.3.2 Tu estilo de tabla

Si siempre haces las tablas igual (por ejemplo, `booktabs`, centradas, letra pequeña y título abajo), deja el asistente así y pulsa Guardar como mi estilo. Desde entonces, cada tabla nueva empieza con esas opciones. Estilo de Pyx vuelve a las de fábrica. El estilo guarda el formato, nunca el contenido.

5.3.3 Pegar una tabla de Excel directamente en el documento

Copia unas celdas de Excel, LibreOffice Calc, Google Sheets, Word o una página web y pégalas en el editor con Ctrl + V: Pyx reconoce filas y columnas y escribe el `tabular` (figura 5.5). Los números se alinean a la derecha, las celdas combinadas se convierten en `\multicolumn`, las negritas

en `\textbf` y los caracteres especiales se escapan. Funciona también con una sola columna. Si preferías el texto tal cual, Ctrl + Z lo deja como se copió.

En **Configuración** → **Editor** → **Pegar tablas** eliges el estilo de líneas (automático: `booktabs` si el proyecto lo carga) y si la tabla debe venir dentro de un entorno `table`, con el cursor esperando en el título.

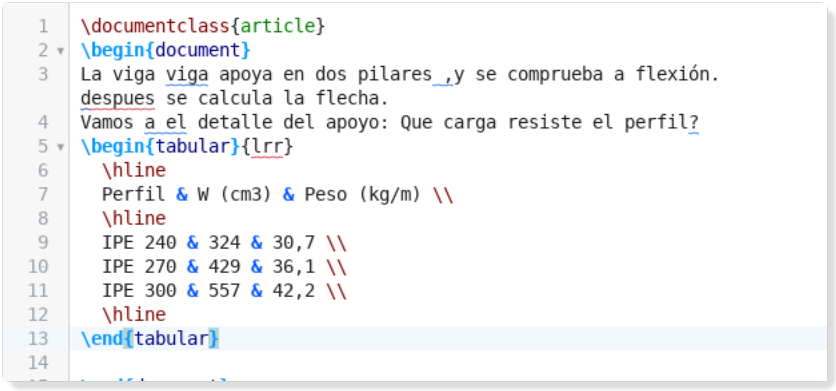


Figura 5.5: Tres filas pegadas desde una hoja de cálculo se convierten en un `tabular`.

5.3.4 Editar una tabla

Con el cursor dentro de un `tabular`, el grupo *Tabla* de la cinta añade o elimina filas y columnas (manteniendo la especificación de columnas), pone o quita `\hline` y cambia la alineación de la columna.

> **Lo que no viaja al pegar** Al pegar, Pyx quita los caracteres invisibles que traen Word, Teams, los PDF y muchas webs (espacios de anchura cero, guiones blandos, marcas de dirección), que rompen un comando o una palabra sin que se vean, y separa las ligaduras de los PDF (*fi* pasa a *fi*). Dentro de una celda de Python, además, los espacios de no separación se vuelven espacios normales.

5.4 Imágenes y figuras

Imagen abre el asistente de figura (figura 5.6):

- Examinar… elige la imagen y escribe su ruta **relativa a la carpeta del documento principal**, que es desde donde LaTeX la busca, subiendo con `..` si está en otra carpeta del proyecto (`../999. Recursos/2. Imagenes/planta.pdf`). Se ve una miniatura de cada imagen.
- El ancho, en porcentaje del texto o en centímetros, un alto máximo (la imagen conserva su proporción) y la rotación.
- + Añadir otra imagen coloca varias una junto a otra, como subfiguras con su propio título: (a), (b)…
- Colocación, pie arriba o abajo y etiqueta. Debajo, el código que se insertará.

Como en las tablas, Guardar como mi estilo hace que las figuras nuevas empiecen con tu formato.

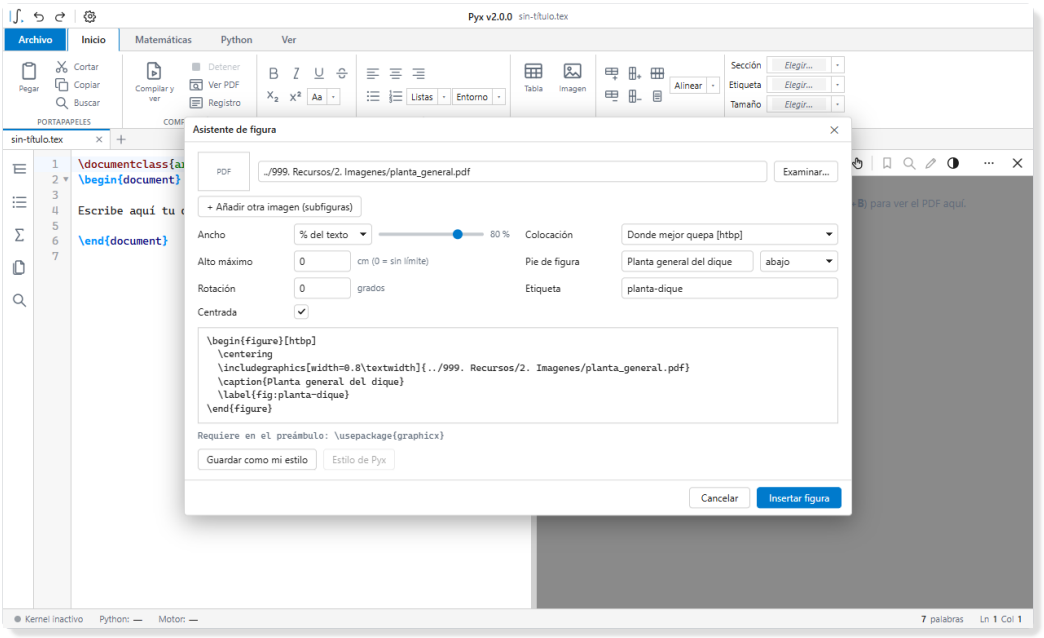


Figura 5.6: El asistente de figura: la ruta queda relativa al documento principal.

Consejo Pega una imagen (una captura, una imagen copiada de otro programa) en el editor con `Ctrl`+`V`: Pyx la guarda en la carpeta `imagenes` del proyecto y escribe la figura con el cursor en el pie. La carpeta se cambia en **Configuración** → **Editor** → **Pegar imágenes**.

5.5 Fórmulas: vista previa al pasar el ratón

Al dejar el ratón sobre una fórmula (`$…$`, `\[ … \]`, `equation`, `align`…) aparece dibujada, sin compilar (figura 5.7). Si la fórmula contiene un `\py{}`, se muestra con su valor actual.

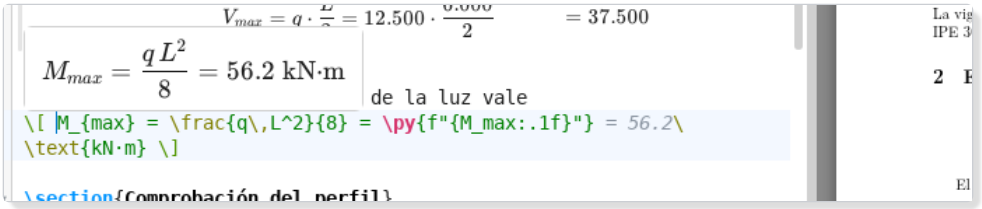


Figura 5.7: Vista previa de una fórmula con el valor calculado por Python.

5.6 Navegar por el documento

`Ctrl`**+clic en una referencia.** Sobre `\ref{fig:viga}`, lleva al `\label{fig:viga}`, aunque esté en otro documento abierto.

`Alt`+`←` **y** `Alt`+`→`**.** Vuelven al sitio donde estabas antes de un salto (un clic lejano, una referencia, un resultado de búsqueda, un salto desde el PDF) y avanzan de nuevo. Funciona también entre documentos.

Marcadores. `Ctrl`+`F2` pone o quita un marcador en la línea; `F2` y `Mayús`+`F2` saltan al siguiente y al anterior.

Plegado. Las flechas del margen pliegan entornos y secciones.

Índice. El panel **TOC** lleva a cualquier sección con un clic.

5.7 Edición inteligente

- Al renombrar `\begin{itemize}`, su `\end{itemize}` cambia a la vez; Ctrl + Z deshace las dos cosas de una vez.
- Intro dentro de una lista añade un `\item` nuevo; detrás de un `\begin{…}` abre el entorno.
- Ctrl + T comenta o descomenta las líneas seleccionadas.
- Alt + ↑ y Alt + ↓ mueven la línea (o la celda entera) arriba o abajo.
- Los paréntesis, corchetes y llaves se cierran solos y se resalta su pareja.
- Ctrl + Alt + D divide el editor en dos paneles con el mismo documento o con documentos distintos.

5.8 Comprobaciones mientras escribes

Pyx subraya los errores de estructura antes de compilar, como el servidor de lenguaje *texlab*: llaves sin cerrar, `\begin` y `\end` que no coinciden, etiquetas repetidas, `\ref` a etiquetas que no existen y `$` sin cerrar. Tras compilar, los errores y avisos del registro también se marcan en su línea. F8 y Mayús + F8 recorren todos los problemas. Si tienes ChkTeX instalado, puedes añadir sus avisos de estilo en **Configuración** → **Compilación**.

5.9 Ortografía, puntuación y redacción

Como en Word, cada tipo de aviso tiene su color, y al dejar el ratón encima se ve el motivo:

Rojo: ortografía. Palabras que el diccionario no reconoce, en español o en inglés (**Configuración** → **Comprobación del lenguaje**). Usa el diccionario de la RAE de LibreOffice ampliado con el vocabulario técnico de la ingeniería (estructuras, hidráulica, ingeniería marítima, geotecnia, estadística) y los nombres de programas habituales, y entiende cómo se forman las palabras en español: los verbos con pronombres pegados (*calculándolo*, *pégalas*), los prefijos (*recalcular*, *semiancho*), los adverbios en *-mente* y los superlativos. Clic derecho sobre una palabra para ver sugerencias o añadirla a tu diccionario.

Azul: puntuación y espacios. Espacios que sobran (dos seguidos, antes de una coma, dentro de un paréntesis) o que faltan (después de una coma o de un punto), signos repetidos, minúscula tras un punto, «a el» y «de el», los signos de apertura ¿ ¡ que faltan y el espacio entre un número y su unidad (*25 kN*, no *25kN*).

Verde: redacción. Palabras repetidas («la viga viga»), expresiones que conviene evitar («en base a», «a nivel de», «de acuerdo a»), redundancias («subir arriba»), «haber» impersonal en plural («habían muchos») y errores frecuentes como «sobretodo», «osea» o «aveces».

Las comprobaciones ignoran los comandos, las fórmulas, el código y los comentarios.

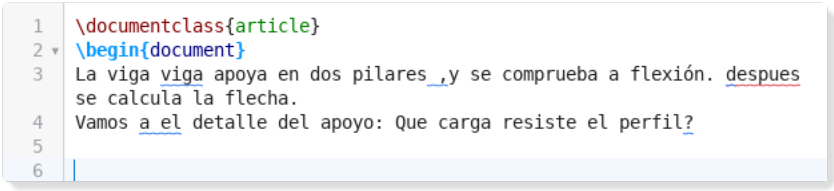


Figura 5.8: Ortografía en rojo y puntuación en azul.

LanguageTool

Para una revisión gramatical completa, Pyx puede usar *LanguageTool*, el motor de LibreOffice. En **Configuración** → **Comprobación del lenguaje** → **Servidor de LanguageTool** escribe:

- `http://localhost:8081` si tienes tu propio servidor (`java -jar languagetool-server.jar --port 8081`); el texto no sale de tu equipo;
- `https://api.languagetool.org` para el servidor público, que recibe el texto del documento.

Sus avisos aparecen como problemas (F8), cada uno con las correcciones propuestas, que se aplican con un clic.

5.10 Buscar y reemplazar

Ctrl + F abre la búsqueda en el documento (o en el PDF, si el foco está en el visor), con mayúsculas, palabra completa y expresiones regulares. Ctrl + R abre directamente el reemplazo: con una palabra seleccionada, la busca en todo el documento, marca todas sus apariciones y deja el cursor en «Reemplazar» para escribir la palabra nueva; Reemplazar todo las cambia de una vez (y Ctrl + Z lo deshace).

5.10.1 Buscar en todo el proyecto

Ctrl + Mayús + F abre el panel **Buscar en el proyecto** (figura 5.9). Busca en todos los archivos de la carpeta del proyecto (`.tex`, `.pltx`, `.bib`, `.py`, `.sty`…) y agrupa los resultados por archivo; un clic abre el archivo con la coincidencia seleccionada. Los documentos abiertos se buscan tal como están en el editor, con los cambios sin guardar.

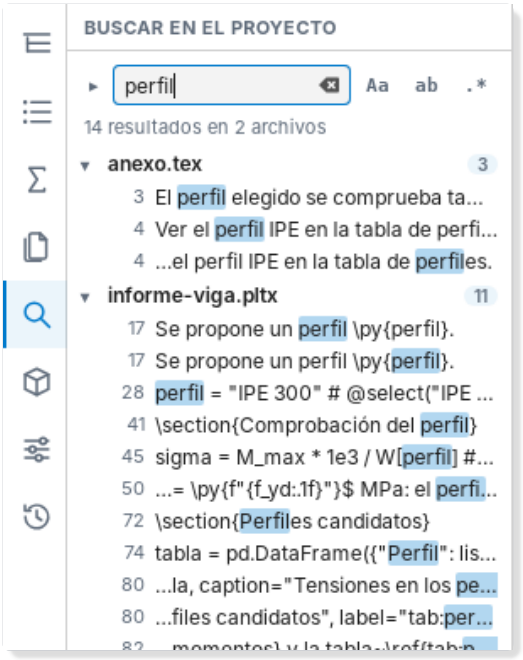


Figura 5.9: Resultados agrupados por archivo.

La flecha despliega el reemplazo. Todo reemplaza en todos los archivos: en los abiertos, el cambio se hace en el editor (se puede deshacer); los demás archivos de texto se guardan directamente, tras confirmarlo.

6

Celdas de Python

Una celda es un bloque de Python dentro del documento. En el archivo se escribe entre dos comentarios de LaTeX:

```
%#python
L = 6.0
q = 12.5
M = q * L**2 / 8
%#end
```

En el editor no se ven esas dos líneas: Pyx dibuja la celda como en Jupyter o VS Code, con su barra de herramientas arriba y la salida debajo. En el PDF la celda no aparece; aparece lo que tú decidas mostrar (capítulo 7).

6.1 Crear y ejecutar celdas

Nueva celda: `Ctrl`+`Alt`+`C` o Nueva celda. Se crea vacía, con el cursor dentro.

Ejecutar la celda: `Ctrl`+`Intro`, o el botón a su izquierda.

Ejecutar y avanzar: `Mayús`+`Intro` ejecuta la celda y lleva el cursor debajo.

Ejecutar todas: `Ctrl`+`Alt`+`Intro` reinicia el espacio de nombres y ejecuta todas las celdas desde el principio.

Interrumpir: el botón que sustituye al mientras la celda se ejecuta, o Interrumpir en la cinta. Las variables calculadas hasta ese momento se conservan. Si la celda no responde, un segundo clic termina el proceso.

La barra de la celda (visible al poner el cursor en ella) tiene, como en VS Code: ejecutar las celdas anteriores, ejecutar esta y las siguientes, dividir la celda por el cursor, más acciones (cortar, copiar, insertar arriba o abajo, borrar la salida, copiar la salida) y eliminar.

> **Consejo** Un clic en la barra azul de la izquierda de una celda la pliega o la despliega; lo mismo con la de su salida. Minimizar todas en la pestaña **Python** pliega todas a la vez.

6.2 Las salidas

Debajo de cada celda aparece lo que produce, en el orden en que lo produce:

- el texto de `print()` y los avisos;

- el valor de la última línea, si es una expresión (un número, una lista, un DataFrame…);
- las tablas de pandas, con formato (figura 6.1);
- las gráficas de Matplotlib, Plotly, Bokeh o Altair (figura 6.2);
- fórmulas en LaTeX (SymPy, handcalcs), HTML, Markdown, imágenes, audio y vídeo.

El número entre corchetes es el contador de ejecución, como en Jupyter, y junto al resultado se indica el tiempo que tardó la celda. Las imágenes y páginas HTML se pueden abrir en una pestaña o en el navegador.



Figura 6.1: Un DataFrame de pandas.

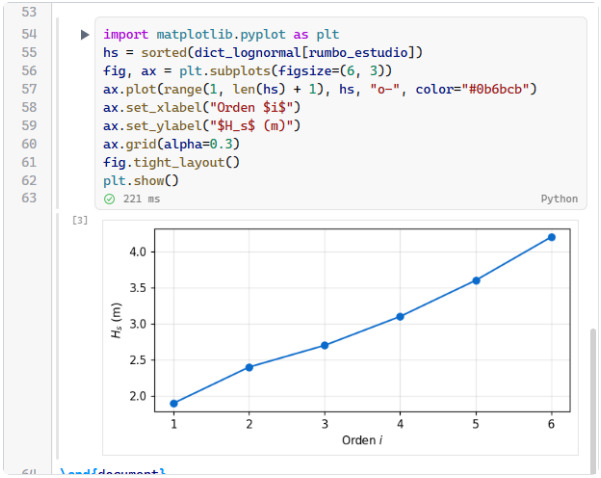


Figura 6.2: Una gráfica de Matplotlib.

> **Nota** Las salidas se guardan dentro del `.pltx`: al reabrir el documento se ven sin ejecutar nada. Si la celda cambia, su resultado se marca como «desactualizado» para que nunca pase por actual un número que ya no corresponde a su fórmula.

6.3 Errores

Si una celda falla, la salida muestra el error con su traza, en color y con los números de línea del documento: un clic en «línea 12» lleva a esa línea. El error aparece también en el panel **Problemas** del registro.

Instalar un paquete que falta

Si el error es `ModuleNotFoundError` (la celda importa una librería que no está instalada), aparece el botón Instalar … (figura 6.3). Pyx instala el paquete en el Python que usa, reinicia el kernel y basta con volver a ejecutar la celda. Además, al abrir un documento, Pyx comprueba qué librerías importan sus celdas y, si falta alguna, lo avisa en la barra de estado.

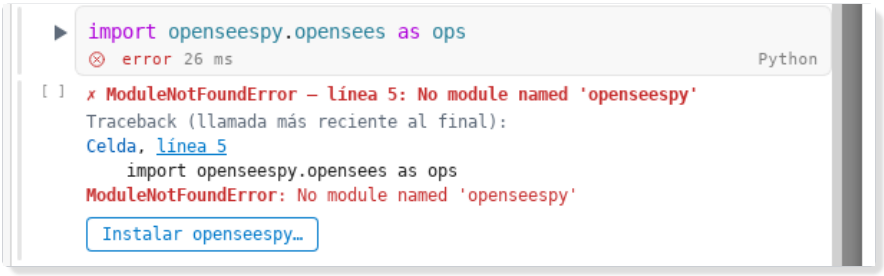


Figura 6.3: Una librería que falta se instala con un clic.

6.4 Autocompletado y ayuda de Python

Dentro de una celda, el autocompletado lo resuelve *jedi* (el mismo motor que usan IPython y JupyterLab) sobre las variables reales del kernel: `np.lin` ofrece `linspace` y `linalg` (figura 6.4); `df.` ofrece los métodos del DataFrame que has creado; dentro de una cadena, como en `pd.read_csv("da`, ofrece los archivos de la carpeta del proyecto.

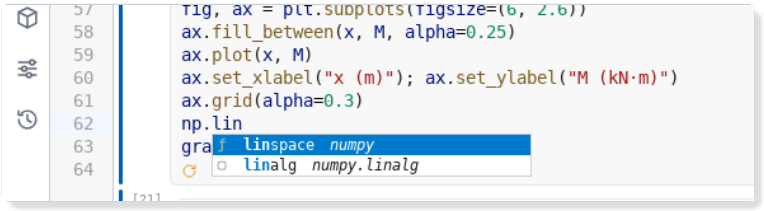


Figura 6.4: Autocompletado con jedi dentro de una celda.

Al escribir el paréntesis de una llamada aparece la **firma** de la función, con el parámetro que estás escribiendo en negrita, y su documentación (figura 6.5). Al dejar el ratón sobre un nombre, se muestra su ayuda. Esc la cierra.

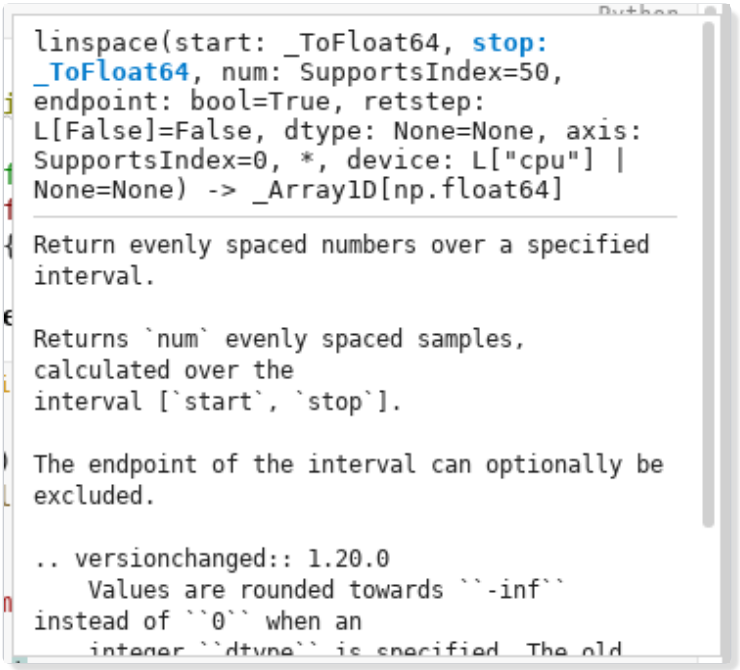


Figura 6.5: La firma de `np.linspace` al escribir el primer argumento.

6.5 El kernel

Las celdas se ejecutan en un intérprete de Python persistente, el *kernel*: las variables y los `import` se conservan entre ejecuciones, como en Jupyter. Las celdas se ejecutan desde la carpeta del documento principal, así que `open("datos.csv")` lee el archivo que está junto al documento.

- Reiniciar (`Ctrl`+`Alt`+`R`) empieza con un espacio de nombres vacío.
- Limpiar salidas borra las salidas de todas las celdas; Ocultar salidas las oculta sin borrarlas (los cálculos siguen yendo al PDF).
- Si una celda modifica una librería (por ejemplo `math.pi = 3`), Pyx lo detecta y, al recompilar desde cero, sustituye el proceso por uno nuevo para que el cambio no contamine el siguiente cálculo.

Pyx admite las magias de handcalcs (`%%render`, `%%tex`), `display()`, los objetos con `_repr_html_`, `_repr_latex_` y similares, `asyncio` y las celdas con `await`.

7

Del cálculo al documento

Las celdas calculan; este capítulo explica cómo llevar sus resultados al PDF. Hay cuatro caminos, de menor a mayor elaboración: un valor en el texto, un texto condicional, una tabla o figura completa, y el desarrollo del cálculo con handcalcs.

7.1 Un valor en el texto: `\py{}`

`\py{expresión}` escribe en el PDF el resultado de cualquier expresión de Python, evaluada con las variables de las celdas:

```
La viga salva una luz de \py{L} m y el momento máximo es
de \py{round(M, 1)} kN·m.
```

Para dar formato a los números se usan las *f-strings* de Python: `\py{f"{M:.2f}"}` escribe dos decimales; `\py{f"{N:,.0f}"}`, separadores de miles; `\py{f"{x:.3e}"}`, notación científica.

Lo que escribe depende de lo que devuelve la expresión:

Un número con todas sus cifras significativas, nunca en notación `1e+05`; un `NaN` o un infinito dan un error en lugar de colarse en el PDF.

Un texto tal cual: es tu propio LaTeX. Por eso una función que devuelve las filas de una tabla (`"1 & 2.5 \\..."`) se coloca dentro de un `tabular`.

Una tabla de pandas (un `DataFrame` o una `Series`) se escribe como un `tabular` completo, como la que se ve bajo una celda: `\py{resumen}` dentro de un entorno `table`. Para más de 500 filas, muestra una parte (`\py{df.head(20)}`) o usa `tabletex`.

Una cantidad de Pint, una expresión de SymPy o cualquier objeto que sepa dibujarse en LaTeX se compone como fórmula.

> **Si una variable recibe dos valores** Cuando dos celdas, o dos archivos vinculados (capítulo 8), dan valor a la misma variable, cada `\py{}` muestra el valor que le dio la celda o el archivo *más próximo por encima*, como al leer un cuaderno de arriba abajo. Un `\py{}` situado antes de todos ellos (un resumen al principio del informe) muestra el valor final.

En el editor, a la derecha de cada `\py{}` se ve en gris su valor actual, sin compilar (al estilo de Mathcad). Esos valores se actualizan al ejecutar celdas y al compilar; se pueden ocultar en **Configuración** → **Celdas Python**.

> **Nota** Mientras escribes, los valores en gris de las expresiones que modificarían datos (por ejemplo `lista.pop()` o `next(iterador)`) no se calculan: dicen «se calcula al compilar». Así, mirar el documento nunca cambia los resultados.

7.2 Un texto u otro: `\pyif`

`\pyif{condición}{si se cumple}{si no}` elige el texto según una condición de Python. El tercer argumento es opcional:

```
El perfil \pyif{sigma <= f_yd}{\textbf{cumple}}{\textbf{no cumple}}
con un aprovechamiento del \py{f"{100*sigma/f_yd:.0f}"} \%.
```

Los dos textos pueden contener cualquier LaTeX, incluso otros `\py{}` y `\pyif`.

7.3 Tablas y figuras completas: el módulo `pyx`

Las celdas pueden importar las ayudas de Pyx con `from pyx import …`; dentro de `\py{}`, `tex`, `texesc`, `tabletex` y `figtex` se usan directamente, sin importarlas:

`figure(nombre)` guarda la figura actual de Matplotlib en la carpeta de imágenes del proyecto (`xref/`) y devuelve su ruta, para usarla en `\includegraphics{\py{ruta}}`.

`figtex(nombre, caption=…, label=…, width=0.8)` devuelve el entorno `figure` completo: se coloca con un solo `\py{}`.

`tex(objeto)` convierte en LaTeX una expresión de SymPy, una cantidad de Pint con sus unidades, una matriz de NumPy o un DataFrame de pandas.

`tabletex(df, caption=…, label=…, prec=…)` devuelve una tabla completa a partir de un DataFrame.

`texesc(texto)` escapa los caracteres especiales de LaTeX de un texto cualquiera.

7.4 Un ejemplo completo

Lo que sigue lo ha calculado una celda de este mismo manual al compilarlo. La celda es:

```
P = 15.0          # carga (kN)
L = 3.0           # vuelo (m)
E = 210e3         # módulo de elasticidad (MPa)
I = 8356e4        # inercia del IPE 300 (mm4)

flecha = P * 1e3 * (L * 1e3)**3 / (3 * E * I)      # mm
admisible = L * 1e3 / 300                           # mm (L/300)
# ... la gráfica con Matplotlib ...
ruta_flecha = figure("flecha")
```

Y el texto del documento es:

```
La flecha en el extremo es \py{f"{flecha:.2f}"} mm, \pyif{flecha <= admisible}
{menor}{mayor} que la admisible de \py{f"{admisible:.1f}"} mm.
```

que en el PDF se convierte en:

La flecha en el extremo es 7.69 mm, menor que la admisible de 10.0 mm. La figura 7.1 muestra la deformada.

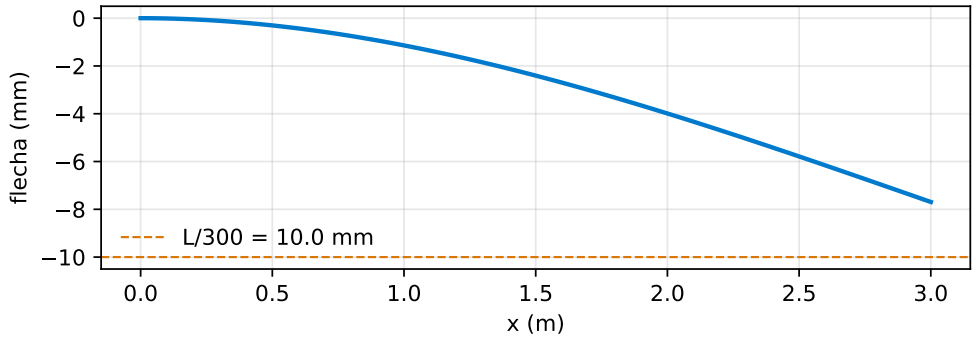


Figura 7.1: Deformada de la ménsula, dibujada por una celda de Python al compilar este manual.

Si mañana cambia la carga a 20 kN, basta con editar la celda y compilar: el número, la comparación y la gráfica cambian a la vez.

7.5 El desarrollo del cálculo: handcalcs

Para que el PDF muestre el cálculo paso a paso, como se haría a mano, usa la biblioteca *handcalcs*: importa `handcalcs.render` en una celda anterior y empieza la celda con `%%render`:

```
%%render
M_max = q * L**2 / 8
V_max = q * L / 2
```

handcalcs escribe cada fórmula con sus valores sustituidos y su resultado, y Pyx la coloca en el PDF en el sitio de la celda. `%%tex` hace lo mismo y, además, imprime el código LaTeX generado. Estas son las únicas celdas que ponen contenido en el PDF por sí mismas; todas las demás solo aparecen a través de `\py{}` y `\pyif`.

Atención Como en Jupyter, `%%render` solo funciona después de `import handcalcs.render`. Si falta, la celda falla con un mensaje que lo explica.

8

Cuadernos de Jupyter y archivos de Python

Muchos cálculos ya existen fuera del documento: en un cuaderno de Jupyter que trabajas en VS Code, o en un archivo `.py`. Pyx los usa tal como están, sin copiar código al informe:

```
\pyfile{calculos/datos.py}
\pynotebook{calculos/oleaje.ipynb}
La altura de ola de diseño es $H_s = \py{Hs}$~m.
```

`\pynotebook{…}` y `\pyfile{…}` son el mismo comando y admiten los dos tipos de archivo; sin extensión, `\pyfile{datos}` busca `datos.py` y `\pynotebook{datos}` busca `datos.ipynb`. La ruta se busca primero en la carpeta del documento principal, como en `\input`, y después en la del capítulo que contiene el comando. En el PDF el comando no escribe nada: sirve para que, desde ese punto, `\py{}`, `\pyif` y las celdas del documento usen las variables del archivo como si fueran suyas, con la misma sintaxis de siempre. `Ctrl`+clic sobre la ruta abre el archivo.

8.1 Qué se puede usar

Todo lo que el cuaderno o el archivo deja definido: números y textos, listas y diccionarios, tablas de pandas (que `\py{}` escribe como tablas de LaTeX), arrays de NumPy y **funciones**. Una función del cuaderno se puede llamar desde el documento, y se lleva con ella todo lo que usa por dentro (otras funciones, diccionarios, módulos como `np`), aunque el documento no lo nombre:

```
\begin{tabular}{ccccc}
  \hline
  $i$ & $H_{s,i}$ (m) & $p_i$ & $Z_i$ & $\ln(H_{s,i})$ \\
  \hline
  \py{filas_ln(rumbo_estudio, len(dict_lognormal[rumbo_estudio]))}
  \hline
\end{tabular}
```

Las imágenes del cuaderno no se traen: una figura se guarda como archivo en el cuaderno y se coloca con `\includegraphics`, como siempre.

8.2 Sin volver a ejecutarlos

Un cuaderno puede tardar minutos en ejecutarse. Pyx no lo repite: toma los valores que ya tiene, por este orden.

1. **De Jupyter, en vivo.** Si el cuaderno está abierto en VS Code, JupyterLab o Jupyter Notebook y ya lo has ejecutado, Pyx copia de ese kernel las variables que usa el documento, en unas décimas de segundo. En el cuaderno no se ejecuta nada ni queda rastro: ni en su historial ni

entre sus variables. Si desde la última compilación no has ejecutado nada en Jupyter, ni siquiera se copian de nuevo.

2. **De los valores guardados.** Con Jupyter cerrado, Pyx usa los últimos valores que tomó, mientras el código del archivo no cambie. También sirven para un `\py{}` nuevo que lea otra variable pequeña del archivo.
3. **Ejecutándolo una vez.** Solo si no hay ninguna de las dos cosas (la primera vez con Jupyter cerrado), Pyx ejecuta el archivo, desde su propia carpeta, como Jupyter. Sus valores quedan guardados y la siguiente compilación ya no lo ejecuta.
4. **Solo lo que ha cambiado.** Si cambias el código del cuaderno con Jupyter cerrado, Pyx recupera todo lo que dejó su última ejecución (aunque sea de otro día) y ejecuta únicamente las celdas modificadas y las que dependen de ellas. Un cuaderno que tardó minutos la primera vez tarda, tras un cambio pequeño, lo que tarde esa celda.

La búsqueda del kernel de Jupyter abierto es instantánea aunque haya muchos archivos de kernels antiguos: Pyx comprueba todos a la vez y recuerda los que ya no responden. **Recompilar desde cero** y Ejecutar todo tampoco vuelven a ejecutar los archivos vinculados. El registro de la compilación dice, para cada uno, de dónde salieron sus valores y cuánto se tardó (figura 10.1), y los avisos señalan las variables que Jupyter todavía no tiene (las de una celda que no has ejecutado allí).

8.3 Se vigilan mientras trabajas

Con *compilar al escribir* activado, Pyx comprueba cada segundo y medio los cuadernos y archivos `.py` del último PDF: si los guardas desde VS Code, si cambian los datos que leen (un Excel, un CSV) o si ejecutas celdas en Jupyter, el PDF se actualiza solo, sin tocar nada en Pyx. Una función de un cuaderno que abre un archivo con una ruta relativa lo busca en la carpeta del cuaderno, la llame quien la llame.

8.4 Archivos `.py` y sus celdas

Un archivo `.py` se ejecuta entero, o por bloques si usa los separadores `# %%` de VS Code, Spyder y PyCharm (también `# In[ ]:`). Con bloques, Pyx vuelve a ejecutar solo los que cambian, como con las celdas del documento; los bloques `# %% [markdown]` son texto y se ignoran.

8.5 Varios archivos y variables con el mismo nombre

Se pueden vincular tantos archivos como se quiera, cada uno en su sitio del documento. Si dos dan valor a la misma variable, cada `\py{}` muestra la del archivo *más próximo por encima*:

```
\pyfile{datos.py}                % Hs = 2.8
La ola de diseño es \py{Hs} m.       % 2.8
\pynotebook{oleaje.ipynb}        % Hs = 2.89 (la media lognormal)
La media de los registros es \py{Hs} m.   % 2.89
```

Las celdas del documento siguen la misma regla: una celda situada después de `\pynotebook{oleaje.ipynb}` lee el `Hs` del cuaderno. El orden en que se usan los archivos es el orden en que aparecen en el documento, que puede ser distinto del orden en que los creaste.

9

Ejecución reactiva

Pyx sabe qué variables define y qué variables usa cada celda. Con esa información construye un *grafo de dependencias*, la misma idea que usa marimo: si cambias una celda, solo hay que volver a ejecutar esa celda y las que dependen de ella, no todo el documento.

9.1 Qué se ejecuta

Imagina un informe con cinco celdas: los datos, la carga de un archivo grande, una simulación que tarda medio minuto, una gráfica y una tabla resumen.

- Si cambias el **texto** del informe, no se ejecuta ninguna celda.
- Si cambias la **gráfica**, solo se ejecuta la gráfica: la simulación no se repite.
- Si cambias los **datos**, se ejecutan las celdas que usan esos datos, directa o indirectamente.

Además, si una celda vuelve a dar exactamente el mismo resultado (por ejemplo, vuelves a escribir `g = 9.81`), las que dependen de ella no se ejecutan: Pyx compara los valores y corta la cadena (*early cutoff*).

El registro de cada compilación indica cuántas celdas se han ejecutado y cuántas había, por ejemplo «Celdas Python : 220 ms (1 ejecutada de 5)».

9.2 Celdas desactualizadas

Cuando editas una celda sin ejecutarla, las celdas que dependen de ella se marcan en cascada con «desactualizada: depende de una celda modificada» (figura 9.1). La celda editada indica «resultado desactualizado». Así ves de un vistazo qué resultados ya no corresponden al código.



Figura 9.1: Tras cambiar la luz en la primera celda, las celdas que la usan se marcan como desactualizadas.

9.3 Mayúsculas+Intro, reactivo

Con la ejecución reactiva activada (es lo habitual), `Mayús`+`Intro` ejecuta la celda y, después, las celdas que dependen de lo que ha cambiado, y solo esas. Si la celda necesita una variable de otra celda que todavía no se ha ejecutado, esa celda se ejecuta primero.

Si prefieres el comportamiento de Jupyter (ejecutar solo la celda pulsada), desactiva **Configuración** → **Celdas Python** → **Ejecución reactiva**. Las celdas que dependen de ella se marcarán entonces como desactualizadas, y la siguiente compilación las pondrá al día. La compilación siempre ejecuta únicamente lo necesario, esté activada o no esta opción.

9.4 Garantías

Ejecutar solo una parte del documento tiene que dar los mismos números que ejecutarlo entero. Pyx lo garantiza así:

- Modificar un objeto cuenta como definirlo: si una celda hace `lista.append(x)`, al editarla se vuelve a crear antes la lista, para que no crezca dos veces.
- Las variables que deja de definir una celda editada o borrada se eliminan, y las celdas que las usaban se vuelven a ejecutar (y fallan, como fallarían en una ejecución completa).
- Los números aleatorios se tratan como un estado común: si una celda usa `np.random`, se repite también la que fijó la semilla.
- Si una celda lee un archivo de datos (un Excel, un CSV) y el archivo cambia, la celda se vuelve a ejecutar aunque su código sea el mismo.

- Lo que el análisis no puede seguir con seguridad (`exec`, `global`, las magias de IPython, `from módulo import *`) hace que Pyx vuelva a la regla clásica: ejecutar desde la primera celda que cambió o, si hace falta, todo.

Nota Esta parte de Pyx se comprueba automáticamente con cientos de documentos generados al azar: para cada uno, ejecutar solo lo que dice el grafo da el mismo resultado, variable por variable, que ejecutar el documento entero.

Consejo **Recompilar desde cero** (`Ctrl` + `Alt` + `Mayús` + `B`) siempre empieza con un espacio de nombres vacío y ejecuta todas las celdas. Es la referencia si alguna vez dudas de un resultado.

10

Compilar

Compilar es convertir el documento en PDF. Pyx hace en una sola acción todo lo que hace falta: guarda, ejecuta las celdas necesarias, sustituye los `\py{}` y `\pyif`, ejecuta LaTeX las veces necesarias y muestra el PDF.

10.1 Las formas de compilar

Compilar y ver (Ctrl + Mayús + B) compila y abre el visor si estaba cerrado.

Detener interrumpe la compilación en curso; se conserva el PDF anterior.

Ver PDF muestra el visor sin compilar.

Registro abre el registro de la compilación: errores, avisos y cuánto tardó cada parte.

Si pulsas Compilar y ver mientras ya compila, Pyx compila una vez más, al terminar, con lo último que hayas escrito: el PDF final es siempre el de tu último cambio. Dos opciones más, fuera de la cinta:

- **Recompilar desde cero** (Ctrl + Alt + Mayús + B): kernel nuevo y sin los archivos auxiliares de compilaciones anteriores.
- **Compilar al escribir**, en **Configuración** → **Compilación**: compila sola cada vez que dejas de escribir un momento (1,1 s por defecto; si el motor tarda más, espera lo que tardó la última vez, como mucho 10 s). Guardar (Ctrl + S) también compila y muestra el PDF, salvo que lo desactives.

Mientras compila, la barra de estado indica la fase: «Preparando…», «Ejecutando Python…», «Componiendo LaTeX…» o «Cargando PDF…». Al terminar indica el resultado y el tiempo total.

10.2 Rápido: el motor precalentado

Al terminar cada compilación, Pyx deja un proceso de LaTeX con el preámbulo (paquetes y fuentes) ya cargado, esperando. Si la siguiente compilación tiene el mismo preámbulo, solo se compone el cuerpo del documento. En una plantilla con muchos paquetes el ahorro es grande: en un informe real con 122 paquetes, el preámbulo solo costaba 7 s de cada compilación. Lo aprovechan Compilar y ver, guardar y la compilación al escribir; solo **Recompilar desde cero** parte de nada. Si cambias el preámbulo, un `.sty` o el motor, Pyx compila de la forma normal.

Escribir el PDF deprisa

XeLaTeX vuelve a comprimir al máximo todo el PDF en cada compilación, figuras incluidas. Pyx le pide una compresión ligera: en el mismo informe, cada pasada baja de 17,3 s a 13,5 s y el PDF solo crece un 3 %. Si vas a entregar el PDF y cada megabyte cuenta, elige **Configuración** → **Compilación** → **Escritura del PDF** → **Archivo lo más pequeño posible**, o guarda una copia con Guardar una copia del PDF.

10.3 Figuras y tablas en su sitio

LaTeX coloca cada figura y cada tabla donde mejor le cabe. Una figura que no entra en lo que queda de página espera a la siguiente, y si no cabe arriba de esa (por ser muy alta) puede acabar *después* del título del apartado siguiente: el texto «se adelanta» a la imagen. No es un fallo de Pyx: el mismo documento, compilado en TeXstudio, da exactamente las mismas páginas.

Con **Configuración** → **Compilación** → **Las figuras y tablas no pasan de su apartado** (activado), antes de cada `\section`, `\subsection` y `\subsubsection` se colocan todas las figuras y tablas pendientes. El orden del PDF es el del documento.

> **Consejo** Para una figura concreta que debe ir *exactamente aquí*, usa `[H]` (paquete `float`) en lugar de `[htbp]`: el asistente de figura lo ofrece en «Colocación». Y evita los `\vspace` negativos alrededor de las figuras: mueven el texto, no la figura.

10.4 Motores

Pyx funciona con **xelatex** (el predeterminado), **pdflatex**, **lualatex** y **Tectonic**. El motor por defecto se elige en **Configuración** → **Compilación** → **Motor LaTeX por defecto**; un documento puede pedir el suyo con una línea `% !TeX program = pdflatex` al principio, o elegirlo en la misma página de Configuración. Con Tectonic no hace falta ninguna distribución de LaTeX: descarga los paquetes que necesita cada documento la primera vez que lo compila.

10.5 Bibliografía, índices y glosarios

Pyx ejecuta solo, entre las pasadas de LaTeX, las herramientas auxiliares que el documento necesita, y solo cuando cambia lo que leen (las mismas reglas que *latexmk*):

- **BibTeX** o **Biber** (con `biblatex`) para las citas;
- **makeindex** para el índice alfabético (`makeidx`, `imakeidx`);
- las listas del paquete **glossaries** (glosario, siglas, símbolos), ordenadas como lo haría `makeglossaries`, sin necesidad de Perl.

También repite LaTeX cuando las referencias cruzadas o el índice cambian, hasta que se estabilizan.

10.6 El registro y los problemas

El botón Registro (pestañas **Inicio** y **Ver**) abre, bajo el editor, el panel de compilación con dos pestañas (figura 10.1):

Problemas los errores y avisos, de LaTeX y de las celdas, con su archivo y su línea; un clic lleva a la línea.

Salida el registro completo de LaTeX y, al final, los tiempos de Pyx: cuánto tardaron las celdas (y cuántas se ejecutaron) y cuánto el motor de LaTeX.

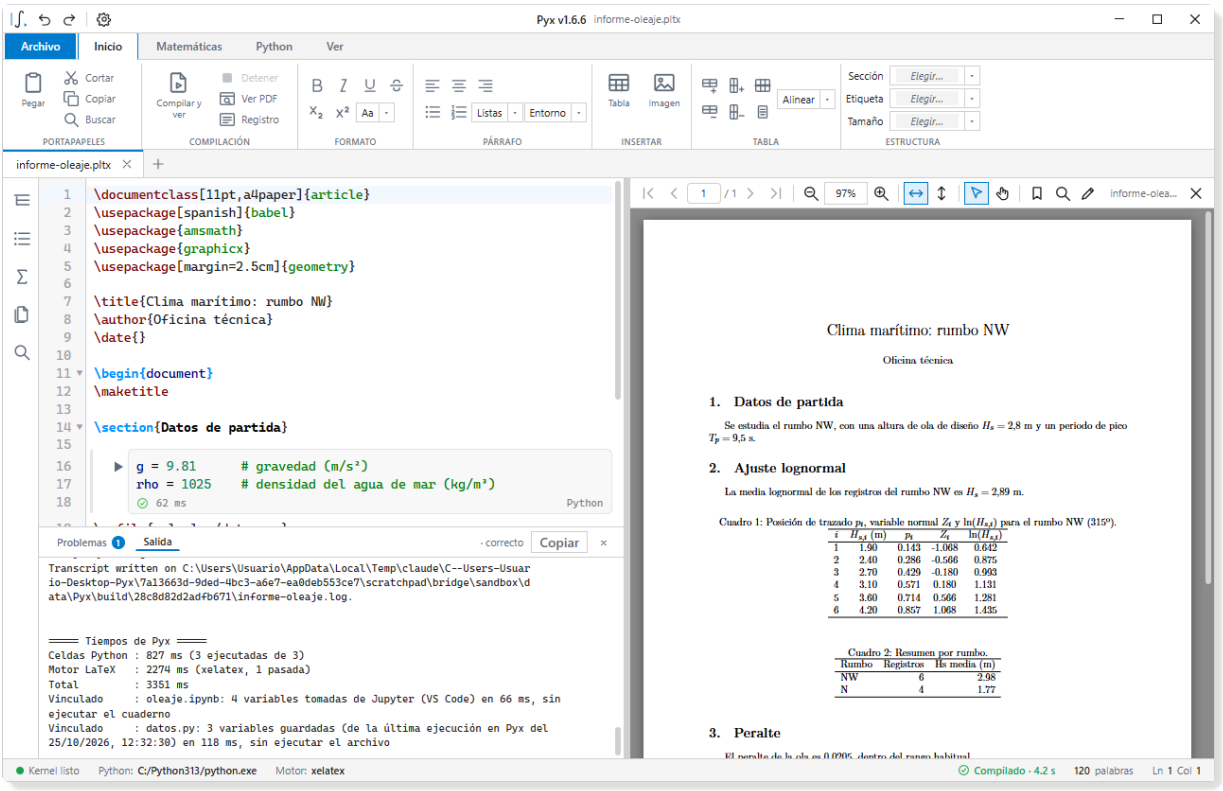


Figura 10.1: La salida de la compilación: al final, de dónde salieron los valores de cada archivo vinculado.

> **Nota** Pyx compila una copia del documento (`.build.tex`) con los valores ya sustituidos, en una carpeta de trabajo fuera del proyecto. Los números de línea de los errores se traducen siempre a los del documento original.

11

El visor de PDF

El PDF se muestra a la derecha del editor y se actualiza solo después de cada compilación, conservando la página y el zoom.

11.1 La barra del visor

Navegación primera página, anterior, número de página (escribe uno para ir a él), siguiente y última.

Zoom alejar, acercar (o `Ctrl`+rueda del ratón) y el porcentaje actual, que vuelve al 100 % con un clic; ajustar al ancho o al alto de la página.

Herramientas seleccionar texto o mover la página con la mano. En cualquier momento, **arrastra con el botón derecho** para mover la página en cualquier dirección (también con la rueda pulsada); con el botón derecho pulsado, la rueda hace zoom. Un doble clic sobre el PDF activa la lupa.

Marcadores el índice del PDF, para ir a cualquier sección (sección 11.5).

Buscar (`Ctrl`+`F` con el foco en el visor) busca en el texto del PDF; `Intro` y `Mayús`+`Intro` recorren los resultados.

Anotar abre la barra de revisión: resaltar, subrayar y tachar texto, lápiz, rotulador, formas, comentarios y la lista de correcciones (sección 11.3).

Contraste oscurece el fondo de las páginas para trabajar de noche.

Ampliar oculta el editor y deja solo el visor.

Ventana auxiliar saca el visor a una ventana propia, ideal para un segundo monitor. Las compilaciones la siguen actualizando; al cerrarla, el visor vuelve a su sitio.

Guardar una copia del PDF y **abrir en la aplicación externa** (el lector de PDF del sistema).

11.2 Rápido con documentos enormes

El visor dibuja primero las páginas que tienes delante, en segundo plano: ni el editor ni el desplazamiento se detienen aunque una página tenga una gráfica con cientos de miles de puntos. Y cuando la vista lleva un momento quieta, dibuja el resto del documento, las páginas más cercanas primero, y las guarda: al desplazarte, las páginas ya están (en un informe real, una página de plano A3 con cincuenta imágenes pasó de tardar 4,5 s en aparecer a 4 ms). Se detiene en cuanto te mueves y nunca ocupa más memoria de la que el equipo puede dar.

> **Consejo** Una gráfica con decenas de miles de puntos en vectorial pesa en cualquier visor y en cada compilación. Si no necesitas ampliar cada punto, guárdala con `rasterized=True` en Matplotlib (los ejes y textos siguen siendo vectoriales) o en PNG a 300 ppp.

11.3 Revisar y corregir el PDF

El botón Anotar de la barra del visor abre las herramientas de revisión. Las marcas se guardan con el PDF del documento, siguen al zoom y sobreviven a las recompilaciones. Mientras tienes una herramienta en la mano, la página no se mueve: ni sola, ni al dibujar con un lápiz digital o con el dedo en una pantalla táctil (que normalmente desplazarían la página).

Resaltar, subrayar y tachar texto. Selecciona el texto con el ratón: al soltar queda marcado exactamente sobre esas palabras, línea a línea. El resaltado se funde con la página como un rotulador de verdad.

Lápiz y rotulador. Trazo libre suavizado. El puntero es un círculo, solo el contorno, del tamaño exacto de la punta.

Línea y flecha. Rectas, curvas o en codo, con punta al final, al inicio, en los dos extremos o sin punta, y trazo continuo o discontinuo. Con Mayús se bloquean a 0°, 45° y 90°.

Recuadro y elipse. Con Mayús, cuadrado y círculo. El recuadro puede tener las esquinas redondeadas.

Cuadro de texto. Un clic donde va y se escribe sobre la página, con su color y su tamaño de letra.

Comentario. Un clic donde va y se escribe al lado de un bocadillo de color. Un clic en el bocadillo **lo pliega** y deja solo el icono; otro lo abre. Un botón de la barra pliega o abre todos.

Borrador. Borra la marca entera que toca o, si lo eliges, solo la parte por la que pasa. Su puntero es un círculo del tamaño del borrador.

Láser. Para señalar algo a quien mira tu pantalla: el trazo brilla un momento y desaparece solo. No se guarda.

Seleccionar y editar. Un clic elige una marca: se mueve arrastrándola o con las flechas del teclado (Mayús: de diez en diez) y muestra sus **tiradores**, como en Figma: los de las esquinas y los lados cambian el tamaño (Mayús conserva la proporción), los extremos de una flecha la reorientan y el tirador del centro de una línea **la curva**. Con la marca elegida, la barra cambia su color, su grosor, su forma y su estilo. Supr la borra y dos clics sobre un texto o un comentario lo editan.

El grosor y el tamaño son números a tu gusto (o con la rueda sobre la casilla). Ctrl+Z y Ctrl+Y deshacen y rehacen. Los botones del final de la barra:

- **Copiar la lista de correcciones**: todas las marcas, página por página, con el texto marcado y cada comentario, lista para pegar en un correo.
- **Guardar el PDF con las anotaciones**: una copia del PDF con las marcas dibujadas en sus páginas, que cualquier lector muestra e imprime.
- **Borrar las de esta página** y **borrar todas** (pide confirmación; Ctrl+Z las recupera).

11.4 Del código al PDF y del PDF al código (SyncTeX)

Del PDF al código: Ctrl+clic sobre una palabra del PDF lleva a la línea del código que la produce, con esa palabra seleccionada.

Del código al PDF: Ctrl+Alt+F (o Ctrl+clic en el editor) lleva al punto del PDF que corresponde a la línea del cursor y lo resalta un momento.

El PDF sigue lo que escribes: con la compilación automática, tras cada compilación el visor se desplaza a lo que estás editando si había quedado fuera de la vista (se puede desactivar en **Configuración** → **Visor PDF**).

Funciona también en proyectos de varios archivos y con Tectonic: Pyx lee directamente el índice de SyncTeX, sin depender de programas externos. Es exacto también debajo de una tabla o una figura escrita con `\py{…}` (que ocupa varias líneas en el PDF y una en el código), y sobre una página sin texto (una figura a página completa, un plano traído con `\includepdf`) lleva a la orden que la puso.

11.5 Marcadores

El botón de marcadores abre, a la izquierda del PDF, su índice (figura 11.1). Si el documento carga el paquete `hyperref`, se muestra el índice del propio PDF, con el número de página de cada entrada y resaltado el apartado que estás leyendo; las ramas se pliegan con su flecha. Si no carga `hyperref`, se muestran las secciones del documento, que Pyx lleva a su sitio en el PDF mediante SyncTeX. Un clic lleva a la sección, también después de recompilar.

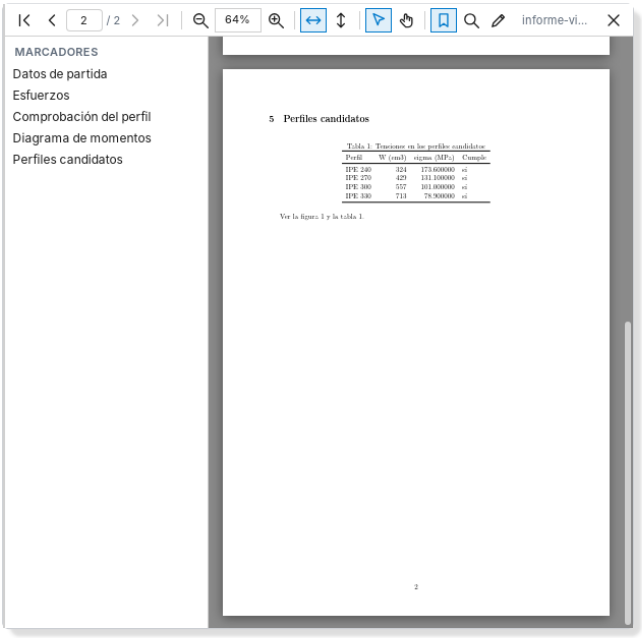


Figura 11.1: Los marcadores del PDF.

12

Macros en Python

Las macros automatizan tareas de edición repetitivas. En TeXstudio se escriben en JavaScript; en Pyx, en Python. Una macro son unas líneas de Python que leen:

`seleccion` el texto seleccionado en el editor;

`texto` el documento completo;

`linea` el número de la línea del cursor;

`variables` las variables calculadas por las celdas del documento (solo lectura);

y dejan en `resultado` el texto que sustituye a la selección. Si `resultado` se queda en `None`, el documento no cambia; lo que se imprima con `print()` se muestra en un aviso.

Pyx trae cinco macros de ejemplo: pasar a mayúsculas, ordenar líneas, convertir líneas en una lista `itemize`, insertar una tabla con las variables numéricas del documento y contar las palabras de la selección.



Figura 12.1: El editor de macros.

Ejecutar: desplegable Macro de la pestaña **Python**, o `Alt`+`Mayús`+`1` a `9` para las nueve primeras.

Crear y editar: Editar macros… en el mismo desplegable. Los cambios se guardan al momento; Ejemplos recupera las de fábrica.

Por ejemplo, esta macro convierte la selección «3,5» en «`\qty{3.5}{\metre}`»:

```
valor = seleccion.strip().replace(",", ".")
resultado = r"\qty{" + valor + r"}{\metre}"
```

Las macros se ejecutan en un espacio de nombres propio dentro del kernel: pueden usar cualquier librería instalada, pero no modifican las variables del documento.

13

Configuración

El engranaje de la barra de título abre **Configuración** (figura 13.1). El cuadro de búsqueda de arriba encuentra cualquier opción por su nombre. Los cambios se aplican al momento. Restablecer esta sección y Restablecer todo vuelven a los valores de fábrica.

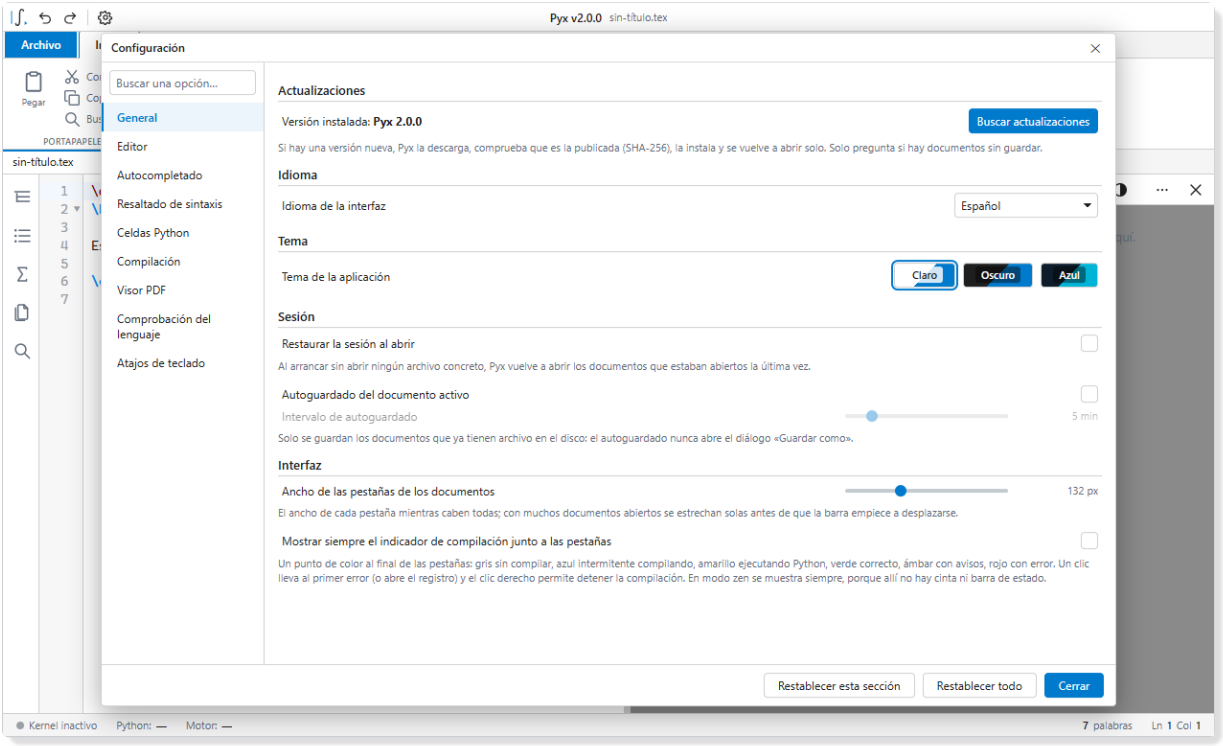


Figura 13.1: La categoría «General» de Configuración, con el botón de actualizar.

General Buscar actualizaciones, idioma de la interfaz (español o inglés), tema, sesión (reabrir los documentos al arrancar y guardado automático) e interfaz (ancho de las pestañas e indicador de compilación).

Editor tipografía, presentación (números de línea, minimapa, guías de sangría, línea activa), ayudas de LaTeX (vista previa de fórmulas, renombrar entornos, marcas del registro), escritura, espacios en blanco, **pegar tablas** (líneas y entorno `table`), **pegar imágenes** (la carpeta donde se guardan) y desplazamiento.

Autocompletado activarlo, mostrarlo al escribir, distinguir mayúsculas, seleccionar la primera opción e iconos.

Resaltado de sintaxis los colores del código LaTeX y de Python, con dos paletas para Python.

Celdas Python el intérprete, el Python de Pyx, los valores en gris de `\py{}`, la ejecución reactiva, el tiempo máximo por celda y guardar los resultados dentro del documento.

Compilación compilar al escribir y su retardo, compilar al guardar, motor por defecto, motor precalentado, escritura del PDF, figuras en su apartado, comprobaciones antes de compilar, codificación de los archivos y Tectonic.

Visor PDF ajuste inicial, el botón que arrastra la página (derecho o central), contraste, la lupa y la sincronización.

Comprobación del lenguaje corrector ortográfico e idioma, puntuación y redacción, y servidor de LanguageTool.

Atajos de teclado cambia cualquier atajo: pulsa Cambiar y teclea la combinación nueva. Los conflictos se marcan en rojo.

14

Atajos de teclado

Los atajos de la aplicación se pueden cambiar en **Configuración** → **Atajos de teclado**; estos son los de fábrica. En macOS, `Ctrl` es `Cmd`.

Atajo	Acción
Archivo	
`Ctrl` + `N`	Nuevo documento
`Ctrl` + `O`	Abrir
`Ctrl` + `S`	Guardar
`Ctrl` + `W`	Cerrar la pestaña
Edición	
`Ctrl` + `Z` / `Ctrl` + `Y`	Deshacer / rehacer
`Ctrl` + `F`	Buscar (en el editor o en el PDF, según el foco)
`Ctrl` + `R`	Reemplazar (la palabra seleccionada, en todo el documento)
`Ctrl` + `Mayús` + `F`	Buscar en todo el proyecto
`Ctrl` + `B` / `Ctrl` + `I`	Negrita / cursiva
`Ctrl` + `T`	Comentar o descomentar
`Ctrl` + `A`	Seleccionar todo (dentro de una celda: su código)
`Ctrl` + `Espacio`	Mostrar el autocompletado
`Alt` + `↑` / `Alt` + `↓`	Mover la línea o la celda
`Alt` + `←` / `Alt` + `→`	Posición anterior / siguiente del cursor
`Ctrl` + `F2`	Poner o quitar un marcador
`F2` / `Mayús` + `F2`	Marcador siguiente / anterior
`F8` / `Mayús` + `F8`	Problema siguiente / anterior
`Ctrl` +clic en `\ref{}`	Ir a su `\label`
`Alt` + `Mayús` + `1` … `9`	Ejecutar una macro
Python	
`Ctrl` + `Intro`	Ejecutar la celda actual
`Mayús` + `Intro`	Ejecutar la celda y avanzar
`Ctrl` + `Alt` + `C`	Nueva celda
`Ctrl` + `Alt` + `Intro`	Ejecutar todas las celdas
`Ctrl` + `Alt` + `R`	Reiniciar el kernel
`Esc`	Cerrar la ayuda de una función
Compilación	
`Ctrl` + `Mayús` + `B`	Compilar y ver
`Ctrl` + `Alt` + `Mayús` + `B`	Recompilar desde cero
Vista	

Atajo	Acción
`Ctrl` + `Alt` + `F`	Ir a esta línea en el PDF
`Ctrl` +clic en el PDF	Ir a ese punto del código
`Ctrl` + `Alt` + `P`	Mostrar u ocultar el visor
`Ctrl` + `Alt` + `D`	Dividir el editor
`Ctrl` + `Alt` + `Z`	Modo zen
`Ctrl` + `Alt` + `T`	Terminal
`Ctrl` + `Alt` + `L`	Registro de compilación
`Ctrl` + `V` con una imagen	Insertarla como figura
Botón derecho + arrastrar en el PDF	Mover la página
`Ctrl` +rueda	Zoom del editor o del PDF

15

Solución de problemas

La barra de estado dice «Python: —» o el kernel está en rojo

Pyx no encuentra un intérprete de Python que funcione. Abre **Configuración** → **Celdas Python** → **Python de Pyx** → **Preparar o actualizar…** para que Pyx prepare el suyo, o elige un intérprete concreto en el desplegable Intérprete de la pestaña **Python**.

«No se detectó LaTeX»

Instala MiKTeX (Windows), MacTeX (macOS) o TeX Live (Linux), o pulsa **Configuración** → **Compilación** → **Descargar Tectonic**. Si acabas de instalar una distribución, reinicia Pyx para que la detecte.

Una celda falla con `ModuleNotFoundError`

La librería no está instalada en el Python que usa Pyx. Pulsa Instalar … en la salida de la celda, o el aviso ! Falta de la barra de estado. Si usas un Python del sistema que no admite instalar paquetes (algunos Linux), haz que Pyx prepare el suyo.

`%%render` no funciona

Añade `import handcalcs.render` en una celda anterior. Si el aviso dice que falta IPython, instálalo (o prepara el Python de Pyx, que ya lo incluye).

Un `\py{}` sale vacío o con un error en el registro

La expresión no se puede evaluar: una variable que no existe (una celda anterior falló o no se ha ejecutado), un error de escritura o una excepción. El panel **Problemas** indica el motivo exacto de cada `\py{}`.

Un cuaderno vinculado se ejecuta otra vez

Pyx solo ejecuta un cuaderno o un archivo `.py` cuando no puede tomar sus valores: Jupyter no lo tiene abierto y no hay valores guardados de ese mismo código. El registro (**Salida**) dice el motivo en la línea «Vinculado». Lo más habitual: el código del cuaderno ha cambiado desde la última vez (guardar el cuaderno tras editar una celda cuenta como cambio). Entonces Pyx recupera lo que dejó su última ejecución y solo vuelve a ejecutar las celdas modificadas y las que dependen de ellas. Para

no ejecutar nada, abre el cuaderno en Jupyter o VS Code y ejecútalo allí: la siguiente compilación tomará los valores de ese kernel.

«Jupyter no tiene …» en los avisos

El documento usa una variable del cuaderno que su kernel de Jupyter todavía no ha calculado: ejecuta en Jupyter la celda que la define y vuelve a compilar.

El resultado no se ha actualizado tras cambiar un archivo de datos

Pyx vuelve a ejecutar una celda si el archivo que leyó con `open()` (o con pandas, NumPy, openpyxl…) ha cambiado. Si lo lees de otra forma, usa **Recompilar desde cero** (Ctrl + Alt + Mayús + B).

Una celda no termina

Pulsa en la celda o Interrumpir en la cinta. Si a los dos segundos no se ha detenido, el botón pasa a Forzar parada: termina el proceso de Python (se pierden las variables) y la siguiente ejecución empieza con un kernel nuevo.

El instalador de Windows no se abre

Si al abrir el instalador Windows dice que no tiene acceso al archivo, es el antivirus: lo ha bloqueado por ser un programa sin firma recién publicado. Permítelo en el antivirus o vuelve a descargarlo pasados unos minutos. Tus documentos y la carpeta de datos de Pyx no se tocan.

Una figura sale después del título del apartado siguiente

Es el comportamiento normal de LaTeX con figuras que no caben (sección 10.3). Comprueba que está activada **Configuración** → **Compilación** → **Las figuras y tablas no pasan de su apartado**, o usa `[H]` en esa figura.

La compilación tarda mucho

- Comprueba en el registro (**Salida**) si el tiempo se va en las celdas o en LaTeX.
- Si son las celdas: con la ejecución reactiva solo se repiten las afectadas; una celda lenta que se ejecuta siempre seguramente lee un archivo que cambia o usa algo que Pyx no puede analizar (`exec`, magias de IPython…).
- Si es LaTeX: comprueba que el motor precalentado está activo; las figuras muy grandes (PNG de varios megas) también ralentizan.

El PDF no se corresponde con lo que he escrito

La barra de estado indica si la última compilación terminó con errores. Si dudas, usa **Recompilar desde cero**, que no reutiliza nada de compilaciones anteriores.

Las ventanas, los menús o el texto se ven pequeños o borrosos

Ajusta el tamaño de la fuente del editor en **Configuración** → **Editor** → **Tipografía**, y el zoom del PDF con `Ctrl`+rueda.

16

Glosario

Archivo vinculado Un cuaderno de Jupyter o un archivo `.py` que el documento usa con `\pynotebook` o `\pyfile`: sus variables están disponibles desde ese punto sin copiar su código.

Celda Bloque de código Python dentro del documento, entre `%#python` y `%#end`.

Compilar Convertir el documento en PDF: ejecutar las celdas necesarias, sustituir los valores y ejecutar LaTeX.

Contenedor `.pltx` El formato de documento de Pyx: código, resultados de las celdas y archivos auxiliares en un solo archivo.

Early cutoff No volver a ejecutar las celdas que dependen de otra si esta ha dado el mismo resultado que antes.

Grafo de dependencias La relación entre las celdas según las variables que definen y usan; decide qué celdas hay que repetir tras un cambio.

jedi Biblioteca de autocompletado de Python que usan IPython, JupyterLab y Pyx.

Kernel El proceso de Python que ejecuta las celdas y conserva sus variables.

Motor precalentado Proceso de LaTeX con el preámbulo ya cargado, esperando la siguiente compilación.

SyncTeX Índice que relaciona cada punto del PDF con su línea de código.

Tectonic Motor de LaTeX autónomo que descarga los paquetes cuando hacen falta.

uv Instalador de Python y de paquetes, muy rápido, que usa Pyx para preparar su propio Python.

Créditos

Pyx se apoya en el trabajo de muchos proyectos de software libre. Entre ellos: Tauri, SolidJS, CodeMirror, PDF.js, pdf-lib, KaTeX, DOMPurify, nspell y los diccionarios Hunspell (RLA para el español, SCOWL para el inglés), uv y python-build-standalone, Tectonic, jedi, handcalcs, placeins, y las ideas de TeXstudio, Jupyter y JupyterLab, marimo, MATLAB, Mathcad, CalcTree, texlab, latexmk, tex-fast-recompile y TeXpresso. El archivo `README.md` del código fuente detalla qué usa Pyx de cada uno.



Este manual se compiló con Pyx el 14 de noviembre de 2026.
Python 3.13.7 · NumPy 2.3.4 · xelatex